21CSS303T - Data Science
Unit 2

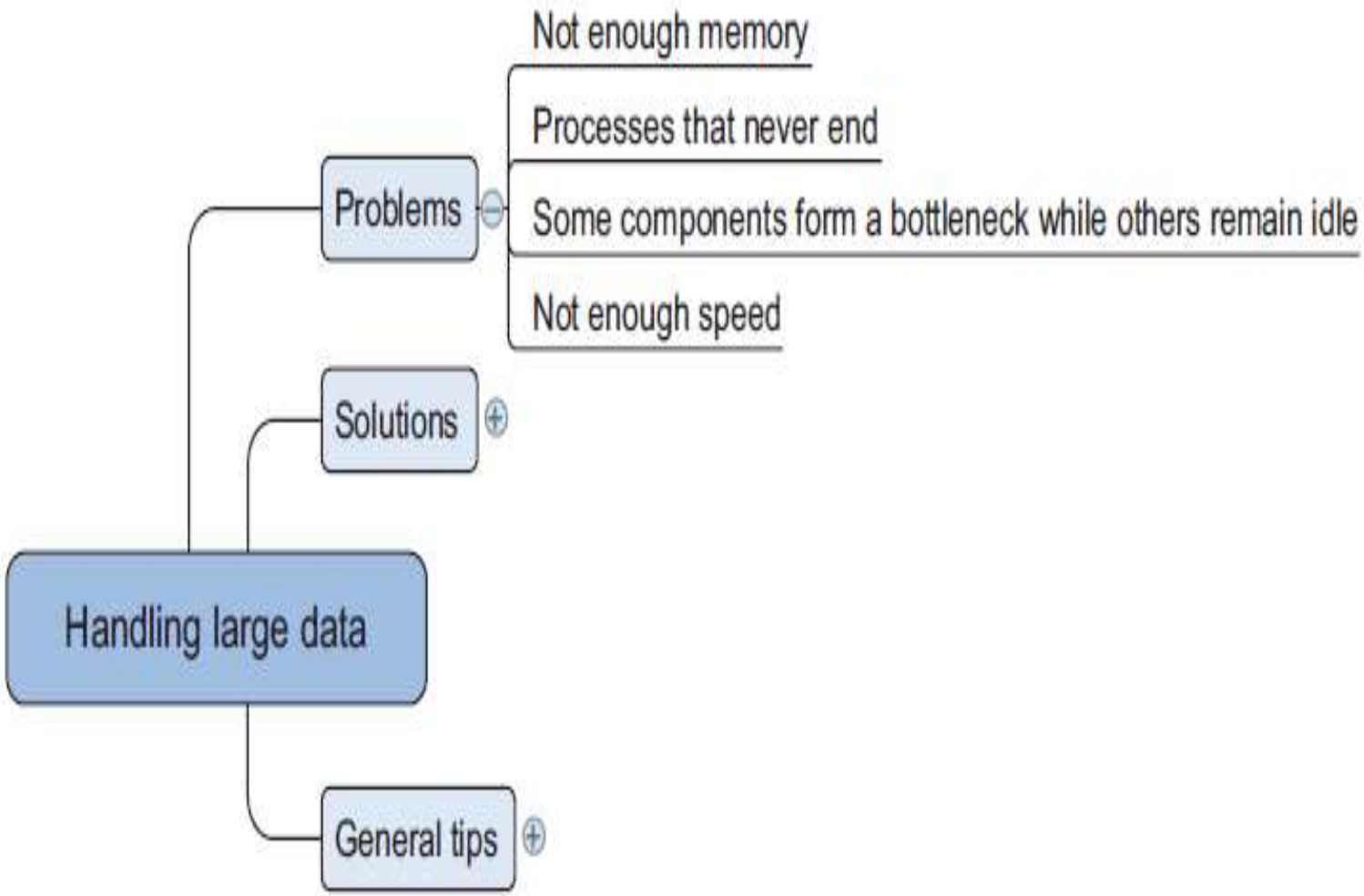
Unit-2: DATA WRANGLING, DATA CLEANING AND PREPARATION

- Data Handling: Problem faced when handling large data- General techniques for handling large volume of data- General programming tips for dealing large data sets
- Data Wrangling: Clean, Transform, Merge, Reshape: Combining and Merging Datasets, Merging on Index, Concatenate, Combining with overlap, Reshaping, Pivoting
- Data Cleaning and Preparation: Handling Missing Data, Data Transformation, String Manipulation, summarizing, Binning, classing and Standardization, outlier/Noise& Anomalies.

Problems faced when handling large data

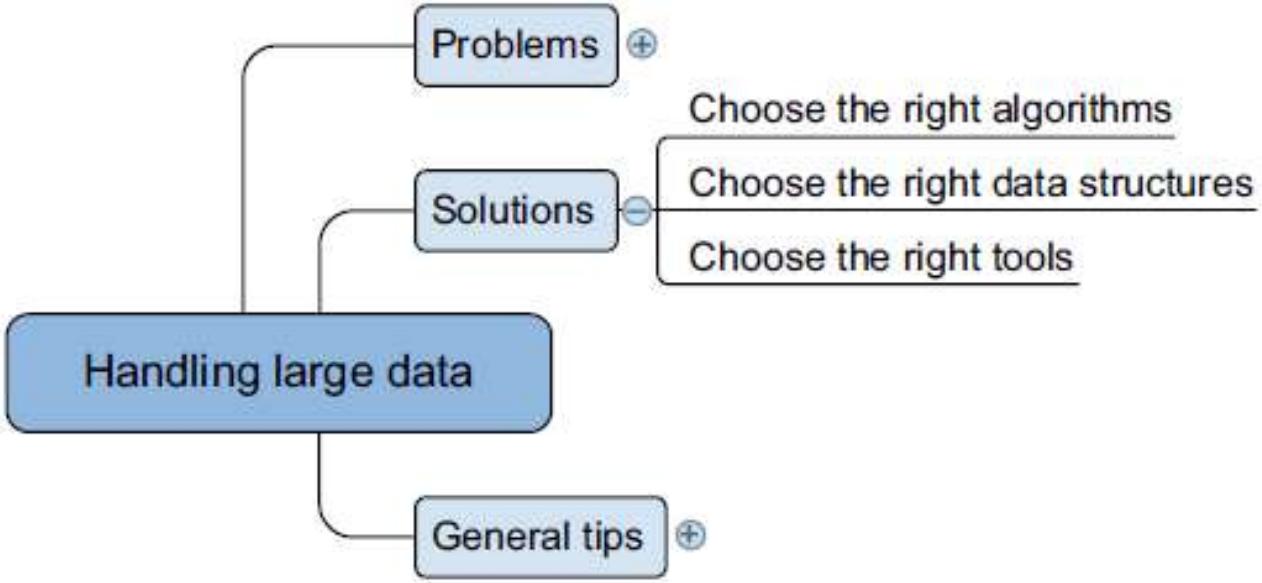
- A large volume of data poses new challenges, such as overloaded memory and algorithms that never stop running.
- It forces you to adapt and expand your repertoire of techniques.
- But even when you can perform your analysis, you should take care of issues such as I/O (input/output) and CPU starvation, because these can cause speed issues.
- Only a few algorithms are designed to handle large data sets; most of them load the whole data set into memory at once, which causes the out-of-memory error.

Problems faced when handling large data



General techniques for handling large volumes of data

- Never-ending algorithms, out-of-memory errors, and speed issues are the most common challenges you face when working with large data.

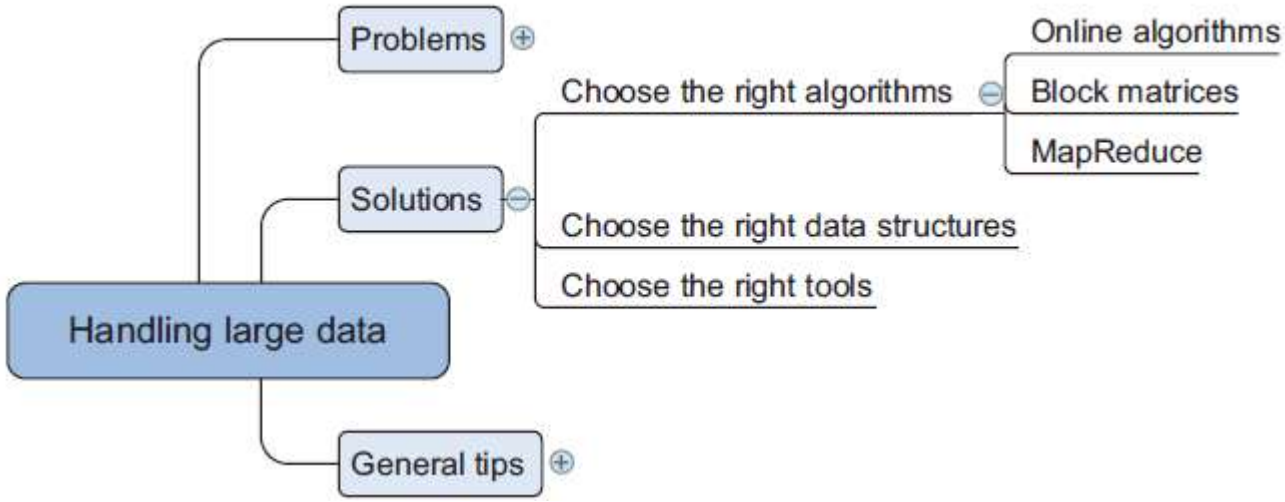


General techniques for handling large volumes of data

- No clear one-to-one mapping exists between the problems and solutions because many solutions address both **lack of memory** and **computational performance**.
- For instance, **data set compression** will help you solve memory issues because the data set becomes smaller. But this also affects **computation speed** with a shift from the slow hard disk to the fast CPU.
- Contrary to RAM (random access memory), the hard disc will store everything even after the power goes down but **writing to disk costs more time than changing information** in the fleeting RAM. When constantly changing the information, RAM is thus preferable over the (more durable) hard disk.

General techniques for handling large volumes of data

- An algorithm that's well suited for handling large data **doesn't need to load the entire data set into memory** to make predictions. Ideally, the algorithm also supports parallelized calculations.



General techniques for handling large volumes of data

ONLINE LEARNING ALGORITHMS

- ***Full batch learning (also called statistical learning)*** — Feed the algorithm all the data at once
- ***Mini-batch learning***—Feed the algorithm a spoonful (100, 1000, …, depending on what your hardware can handle) of observations at a time.
- ***Online learning***—Feed the algorithm one observation at a time.

Online / Incremental Learning Algorithms

These algorithms learn one sample at a time or in mini-batches.

Stochastic Gradient Descent (SGD)

- Updates model for each observation.
- Used in large-scale regression & classification.
- Very memory efficient.
- It is used in Logistic Regression (SGD version) and Linear Regression (SGD version)

Perceptron

- Online classification algorithm.
- Updates weights per instance.
- Works well for streaming data.

Passive-Aggressive Algorithms

- Designed for large-scale classification.
- Efficient for text classification.

Tree-Based Algorithms

Some tree algorithms are optimized for scalability.

Random Forest (parallel implementation)

- Trees can be built independently.
- Easily parallelizable.

XGBoost

- Optimized gradient boosting.
- It Supports Parallel computation and Out-of-core computation
- It is Used in big data competitions.

LightGBM

- Uses histogram-based learning.
- Handles very large datasets efficiently.
- Faster than traditional gradient boosting.

Distributed Algorithms (Big Data Frameworks)

Designed to run on clusters.

MapReduce-based Algorithms

- Used in Hadoop.
  - Distributed K-Means
  - Distributed Logistic Regression

Spark MLlib Algorithms

Scalable implementations of:

- K-Means
- Linear Regression
- Decision Trees
- ALS (Collaborative Filtering)

Large-Scale Clustering Algorithms

Mini-Batch K-Means

- Processes small batches instead of full dataset.
- Faster than standard K-Means.
- Memory efficient.

BIRCH

- Designed for very large datasets.
- Incrementally builds clustering structure.

Algorithms Using Sparse Data Structures

Used when data contains many zeros (like text data).

Naive Bayes

- Works well with high-dimensional sparse data.
- Very fast for large text datasets.

Linear SVM (with SGD)

- Efficient for large-scale classification.
- Works well with sparse matrices.

Deep learning and Approximation

Deep Learning

Designed for massive datasets.

Neural Networks (Mini-batch training)

- Use GPUs.
- Work with large-scale image/text data.

Approximation Algorithms

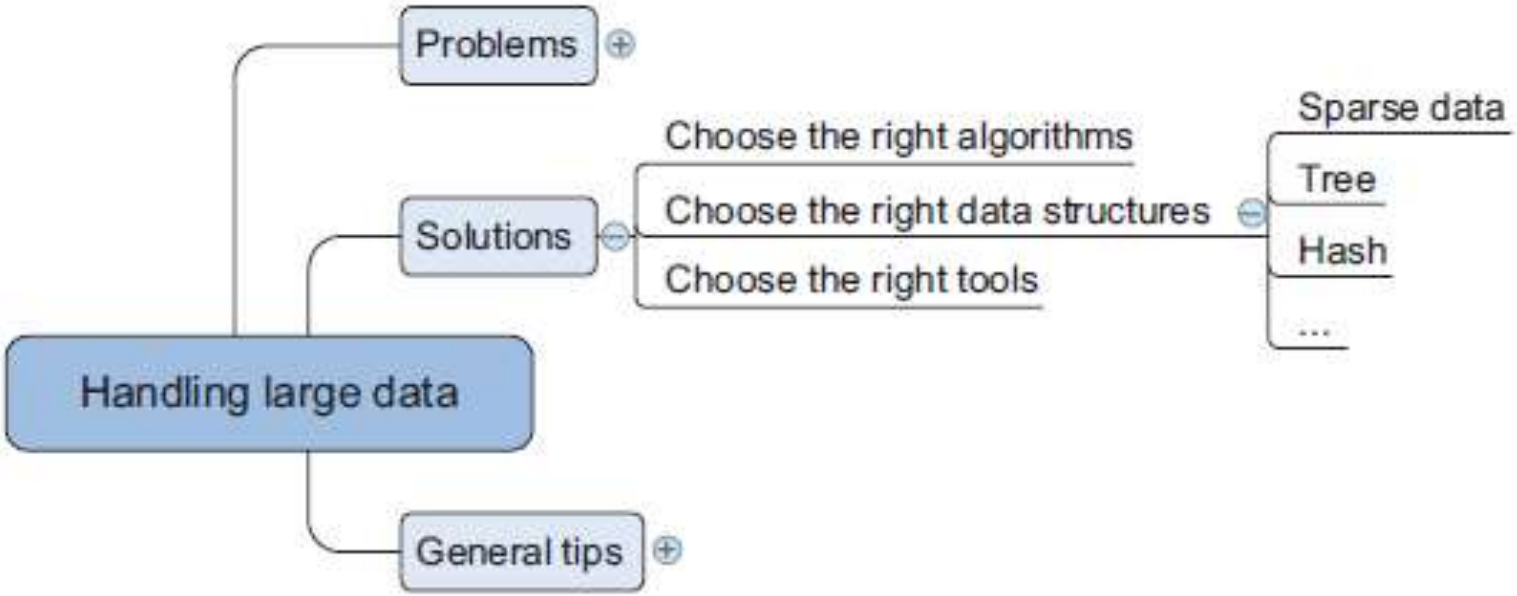
- Instead of exact solution → approximate solution.
- Example:
- Approximate Nearest Neighbor
- Bloom Filters
- Used when:

  Exact computation is too costly.

Algorithms- Comparison

Category	Example Algorithms	Why Suitable for Large Data
Online Learning	SGD, Perceptron	No full data loading
Mini-Batch	Mini-Batch K-Means	Memory efficient
Tree-Based	XGBoost, LightGBM	Parallel support
Distributed	Spark MLlib, MapReduce	Cluster processing
Sparse Data	Naive Bayes, Linear SVM	Efficient with high dimensions
Deep Learning	Neural Networks	GPU + mini-batch

General techniques for handling large volumes of data



General techniques for handling large volumes of data

- **DIVIDING A LARGE MATRIX INTO MANY SMALL ONES**
  - **bcolz -** store data arrays compactly and uses the hard drive when the array no longer fits into the main memory
  - **Dask -** optimize the flow of calculations and makes performing calculations in parallel easier
- **MAPREDUCE – easy to parallelize and distribute**
- MapReduce algorithms are easy to understand with an analogy: Imagine that you were asked to count all the votes for the national elections. Your country has **25 parties, 1,500 voting offices, and 2 million people**. You could choose to
  - gather all the voting tickets from every office individually and count them centrally
  - ask the local offices to count the votes for the 25 parties and hand over the results to you, and you could then aggregate them by party.

Choosing the right data structure

- Algorithms can make or break your program, but the way you store your data is of equal importance.
- Data structures have different storage requirements, but also influence the performance of ***CRUD*** (create, read, update, and delete) and other operations on the data set.

Sparse Matrix

- **SPARSE DATA**
- A sparse data set contains relatively little information compared to its entries (observations).

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 4.6 Example of a sparse matrix: almost everything is 0; other values are the exception in a sparse matrix

Trees

TREE STRUCTURES

- Trees are a class of data structure that allows you to **retrieve information much faster** than scanning through a table.
- A tree always has a root value and subtrees of children, each with its children, and so on.

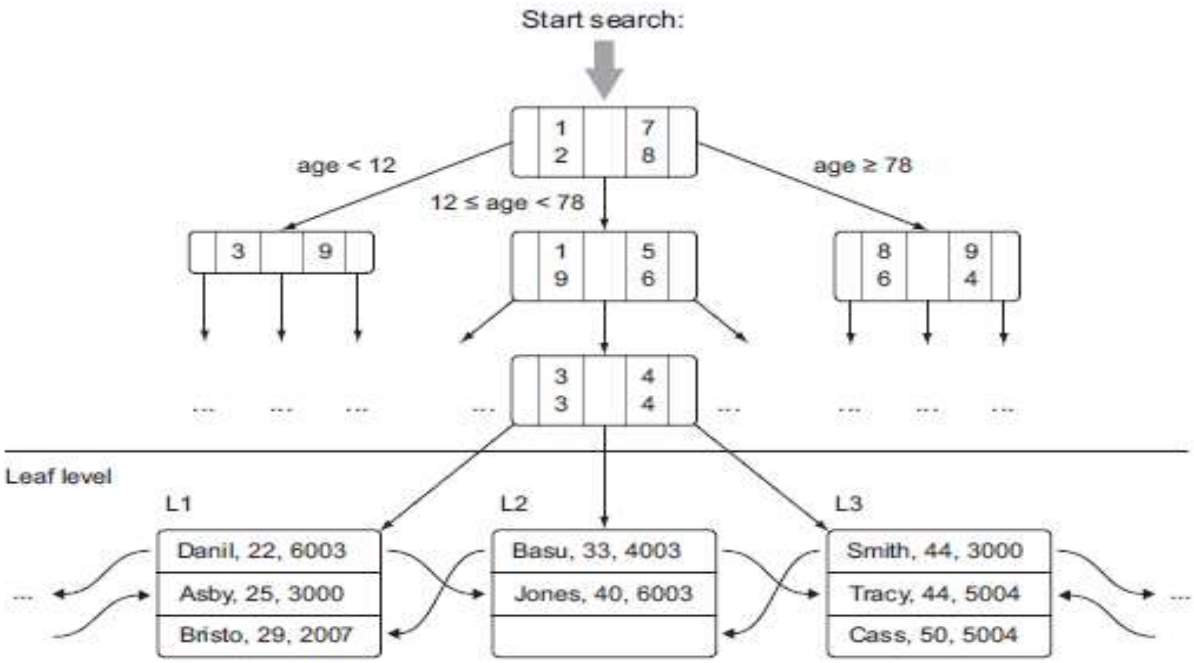
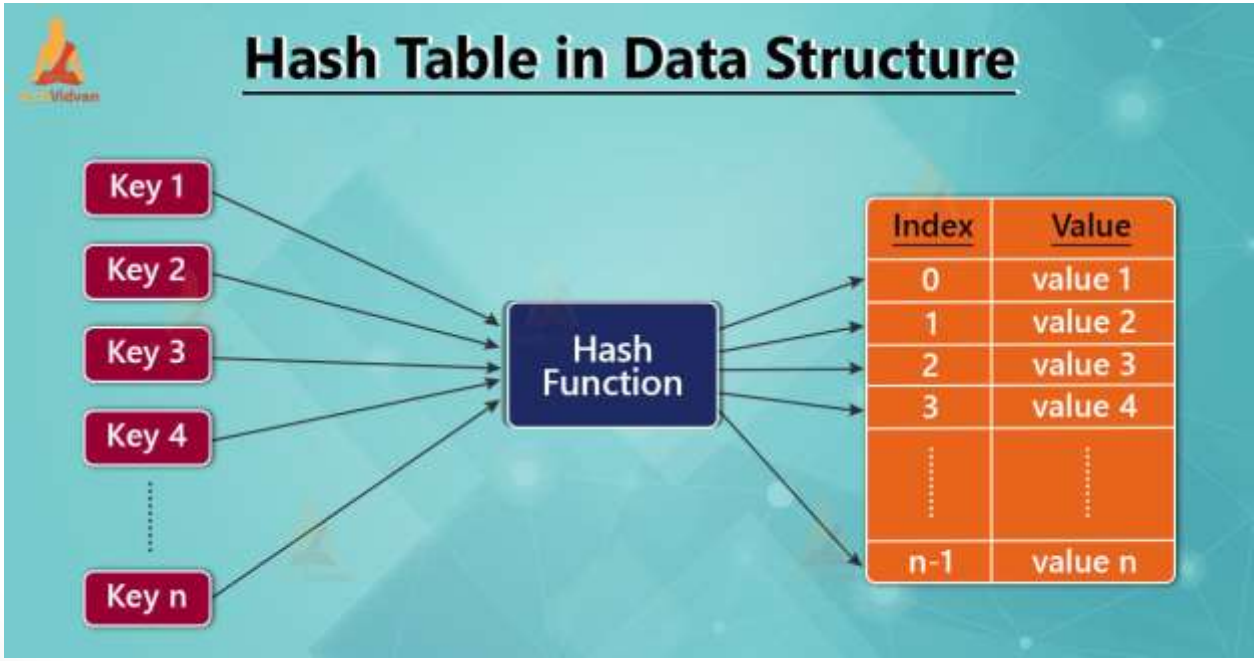


Figure 4.7 Example of a tree data structure: decision rules such as age categories can be used to quickly locate a person in a family tree

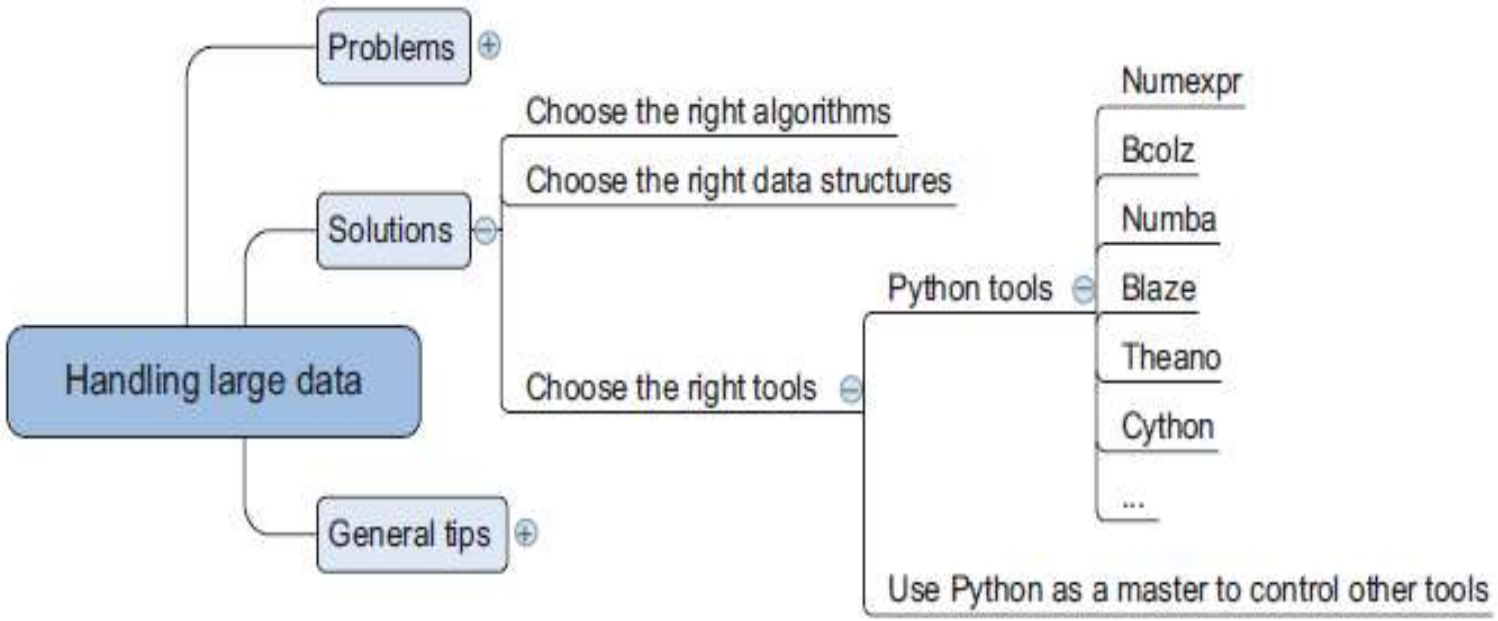
Hash Tables

- Hash tables are data structures that calculate **a key for every value in your data and put the keys in a bucket**.
- This way you can quickly retrieve the information by looking in the right bucket when you encounter the data.
- Dictionaries in Python are a **hash table** implementation, and they're a close relative of **key-value** stores.



Selecting the right tool

- With the right class of algorithms and data structures in place, it's time to choose the right tool for the job.
- The right tool can be a Python library or at least a tool that's controlled from Python.



Selecting the right tool

- Python has a number of libraries that can help you deal with large data.
- They range from smarter data structures over code optimizers to just-in-time compilers.
  - ***Cython***
  - ***Numexpr***
  - ***Numba***
  - ***Bcolz***
  - ***Blaze***
  - ***Theano***
  - ***Dask***

Selecting the right tool

- Python has a number of libraries that can help you deal with large data. They range from smarter data structures over code optimizers to just-in-time compilers.

Cython

- specify the data type while developing the program. Once the compiler has this information, it runs programs much faster.
- Cython is used to speed up Python code by compiling it into C language.
- Python is slower than C. Cython converts Python code into C code, which runs faster.

Example

 # normal python

```
def square(n):
return n*n
```

In Cython, we can add type declarations:

```
cpdef int square(int n):
return n*n
```

Runs much faster than normal Python.

Used in:

scientific computing

machine learning libraries

Selecting the right tool

2. Numexpr

Numexpr is used to speed up numerical calculations on large arrays.

It works well with NumPy arrays.

Example

Normal NumPy:

```
import numpy as np
a = np.arange(1000000)
b = np.arange(1000000)
c = a + b * 2
```

Using Numexpr:

```
import numexpr as ne
c = ne.evaluate("a + b * 2")
```

✓ Faster

✓ Uses multiple CPU cores

3. Numba

Numba is a Just-In-Time (JIT) compiler for Python.

It converts Python functions into fast machine code while running the program.

Example

```
from numba import jit
@jit
def add(a,b):
return a+b
print(add(5,10))
```

✓ **Makes loops and calculations much faster**

Used in:

scientific computing

simulations

machine learning

Selecting the right tool

4. **Bcolz**

Bcolz is used to store large data in compressed form.

It allows you to work with datasets larger than RAM.

Example

```
import bcolz
c = bcolz.arange(10000000)
print(c.sum())
```

✓ Compresses data

✓ Saves memory

5. Blaze

Blaze is used to analyze large datasets from different sources.

It works with:

CSV files

SQL databases

big data systems

Example

```
from blaze import Data
data = Data('data.csv')
data.amount.sum()
```

✓ Provides a simple interface for big data

Selecting the right tool

6. Theano

Theano is used for deep learning and mathematical computations.

It was developed for building neural networks.

Example

```
import theano
import theano.tensor as T
x = T.dscalar('x')
y = x * x
f = theano.function([x], y)
print(f(5))
```

Output:

25

✓ Used in early deep learning frameworks.

7. Dask

Dask is used to process very large datasets using parallel computing.

It can work with large NumPy, Pandas, and machine learning datasets.

Example

```
import dask.array as da
a = da.arange(10000000, chunks=100000)
print(a.sum().compute())
```

✓ Processes data in parallel

✓ Can handle datasets larger than memory

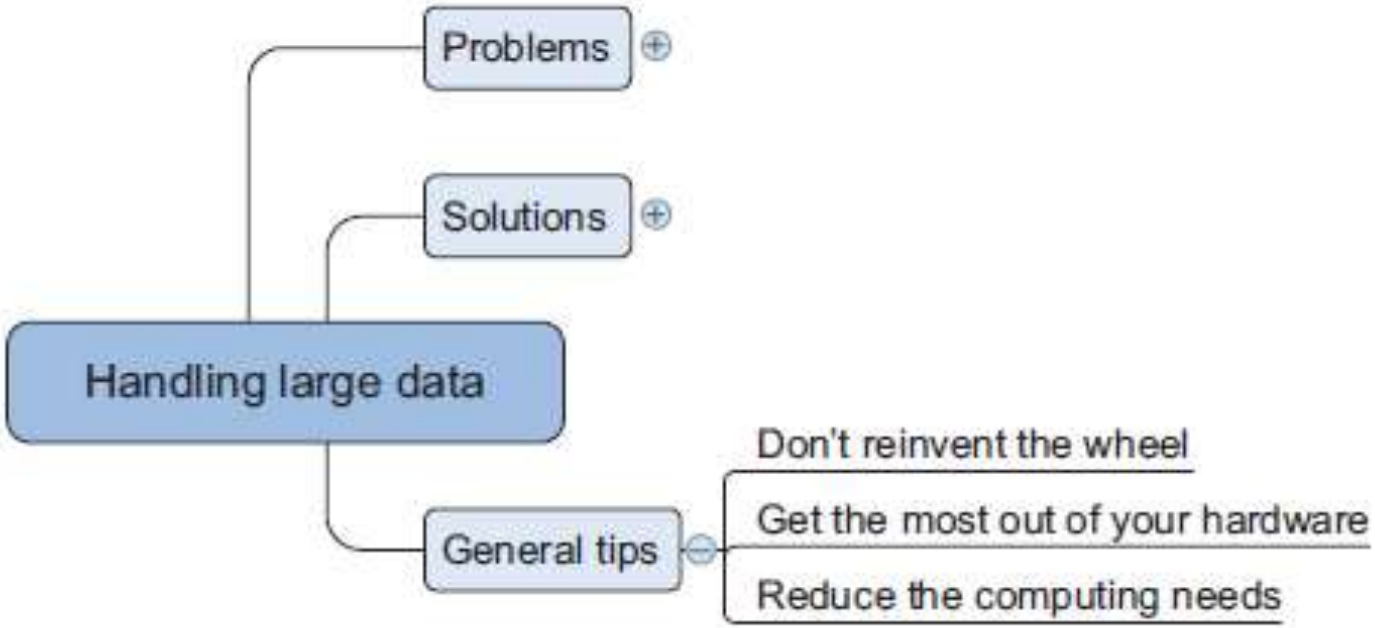
Selecting the right tool

Library	Main Use
Cython	Convert Python to fast C code
Numexpr	Fast numerical array calculations
Numba	JIT compiler to speed Python
Bcolz	Compressed large data storage
Blaze	Query large datasets
Theano	Deep learning computations
Dask	Parallel computing for big data

General programming tips for dealing with large data sets

- *Don't reinvent the wheel.* Use tools and libraries developed by others.
- *Get the most out of your hardware.* Your machine is never used to its full potential; with simple adaptions you can make it work harder.
- *Reduce the computing need.* Slim down your memory and processing needs as much as possible.

General programming tips for dealing with large data sets



General programming tips for dealing with large data sets

- ***Exploit the power of databases.***
  - The first reaction most data scientists have when working with large data sets is to prepare their **analytical base tables** inside a database. This method **works well** when the features you want to prepare are **fairly simple**. When this preparation involves advanced modeling, find out if it's possible to employ **user-defined functions and procedures**.
- ***Use optimized libraries.***
  - Creating libraries like Mahout, Weka, and other machine learning algorithms requires time and knowledge.
  - They are **highly optimized** and incorporate best practices and state-of-the art technologies.
  - Spend your time on getting things done, not on reinventing and repeating others people's efforts, unless it's for the sake of understanding how things work.

General programming tips for dealing with large data sets

- **Get the most out of your hardware**
  - ***Feed the CPU compressed data -*** Avoid CPU starvation is to feed the CPU compressed data instead of the inflated (raw) data.
  - ***Make use of the GPU -*** Sometimes your CPU and not your memory is the bottleneck. If your computations are parallelizable, you can benefit from switching to the GPU. This has a much higher throughput for computations than a CPU.
  - ***Use multiple threads -*** It's still possible to parallelize computations on your CPU. You can achieve this with normal Python threads.

General programming tips for dealing with large data sets

Reduce your computing needs

- "Working smart + hard = achievement."
- This also applies to the programs you write.
- The best way to avoid having large data problems is by **removing as much of the work as possible up front** and letting the computer work only on the part that can't be skipped.

General programming tips for dealing with large data sets

Reduce your computing needs

- ***Profile your code and remediate slow pieces of code*** – Profiler to detect slow parts inside your program
- ***Use compiled code whenever possible, certainly when loops are involved*** - Use functions from packages that are optimized for numerical computations
- ***Otherwise, compile the code yourself*** - just-in-time compiler or implement the slowest parts of your code in a lower-level language such as C or Fortran and integrate
- ***Avoid pulling data into memory*** - reading data in chunks and parsing the data on the fly. This won't work on every algorithm but enables calculations on extremely large data sets.
- ***Use generators to avoid intermediate data storage*** - Generators help you return data per observation instead of in batches.
- ***Use as little data as possible***. If no large-scale algorithm is available and you aren't willing to implement such a technique yourself, then you can still train your data on only a sample of the original data.
- ***Use your math skills to simplify calculations as much as possible***.

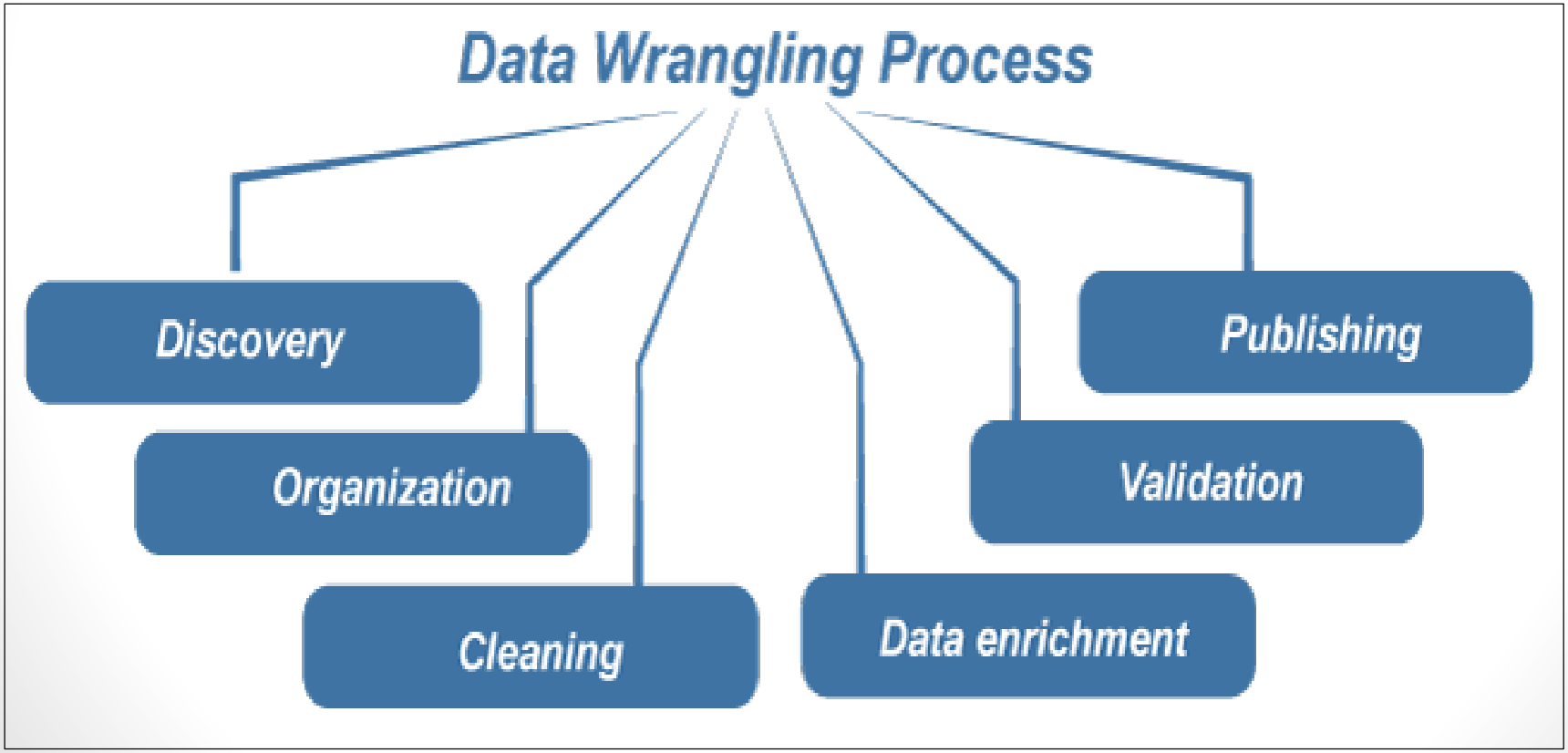
Data Wrangling

- Data Wrangling is referred to as ***data munging*.**
- It is the process of transforming and mapping data from one "raw" data form into another format to make it more appropriate and valuable for various downstream purposes such as analytics.
- The goal of data wrangling is to **assure quality and useful data**

Data Wrangling

Data Wrangling

- **Data Wrangling** is one of those technical terms that are more or less self-descriptive.
- The term **"wrangling"** refers to rounding up information in a certain way.



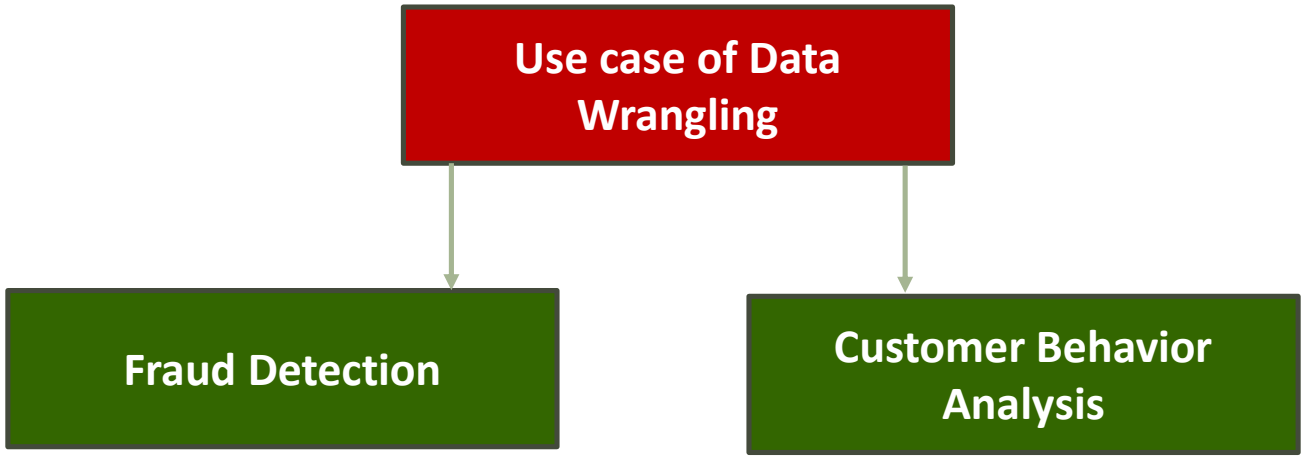
Data Wrangling

- Discovery: Before starting the wrangling process, it is critical to think about what may lie beneath your data.
- Organization: After you've gathered your raw data within a particular dataset, you must structure your data.
- Cleaning: When your data is organized, you can begin cleaning your data. Data cleaning involves removing outliers, formatting nulls, and eliminating duplicate data.

Data Wrangling

- Data enrichment: This step requires that you take a step back from your data to determine if you have enough data to proceed.
- Validation: After determining you gathered enough data, you will need to apply validation rules to your data. Validation rules, performed in repetitive sequences, confirm that data is consistent throughout your dataset.
- Publishing: The final step of the data munging process is data publishing. Data providing notes and documentation of your wrangling process and creating access for other users and applications.

Data Wrangling



Data Wrangling

Data munging is used for diverse use-cases as follows:

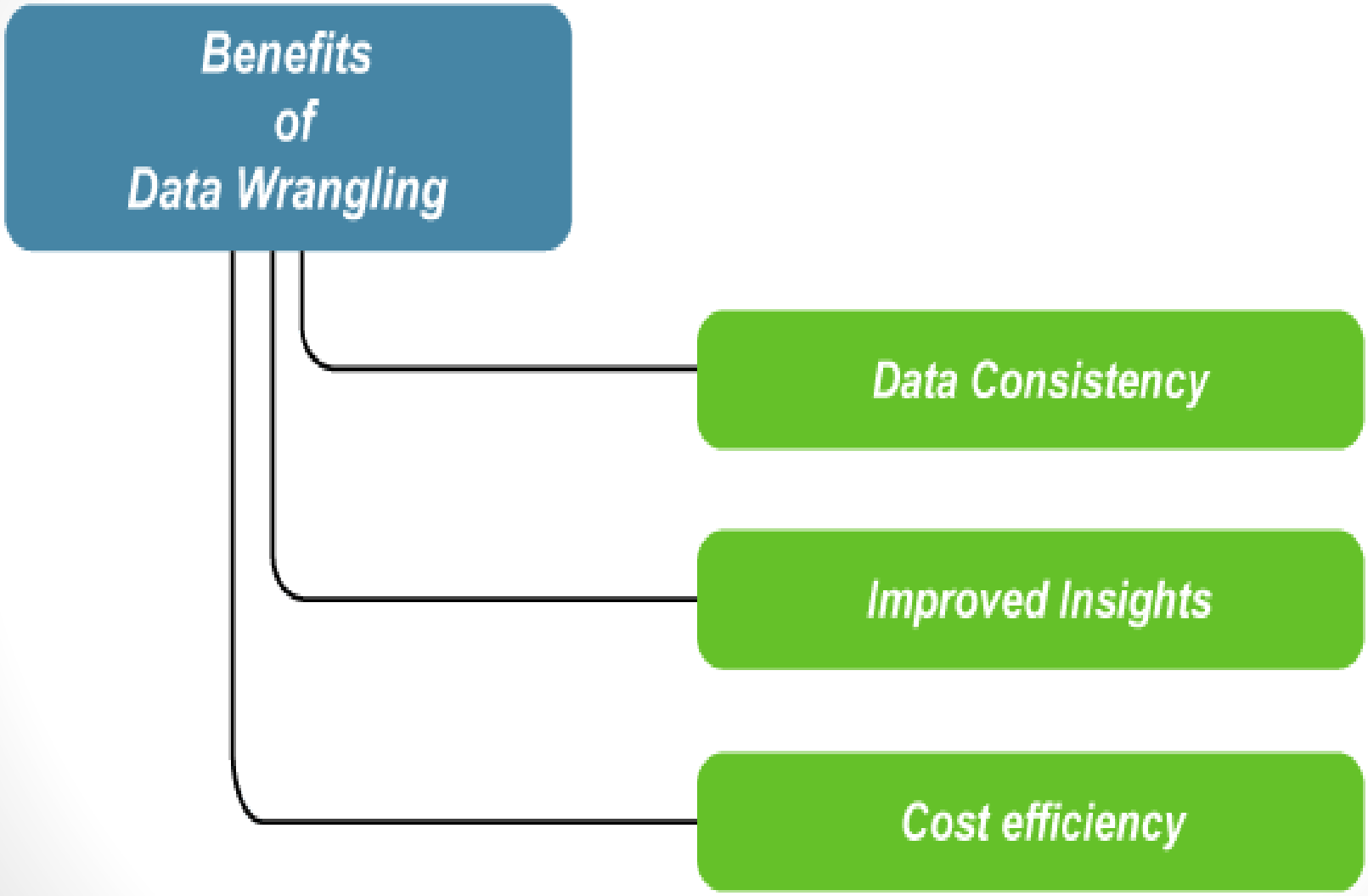
- Fraud Detection - Distinguish corporate fraud by identifying unusual behavior by examining detailed information like multi-party and multi-layered emails or web chats.
- Customer Behaviour Analysis - A data-munging tool can quickly help your business processes get precise insights via customer behavior analysis.

Data Wrangling Tools

There are different tools for data wrangling that can be used for gathering, importing, structuring, and cleaning data before it can be fed into analytics and BI apps.

- **Spreadsheets / Excel** Power Query is the most basic manual data wrangling tool.
- **OpenRefine** - An automated data cleaning tool that requires programming skills
- **Tabula** - It is a tool suited for all data types
- **Google DataPrep** - It is a data service that explores, cleans, and prepares data
- **Data wrangler** - It is a data cleaning and transforming tool
- **Plotly** (data wrangling with Python) is useful for maps and chart data.

Benefits of Data Wrangling



Benefits of Data Wrangling

- Data consistency: The organizational aspect of data wrangling offers a resulting dataset that is more consistent.
- Improved insights: Data wrangling can provide statistical insights about metadata by transforming the metadata to be more constant.
- Cost efficiency: Data-wrangling allows for more efficient data analysis and model-building processes, businesses will ultimately save money in the long run.

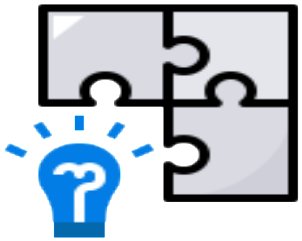
Clean, Transform and Merge

When working with datasets in pandas, you typically follow three major steps:

1. Cleaning – Handling missing values, duplicates, and inconsistent data
2. Transforming – Changing data formats, applying functions, feature engineering
3. Merging – Combining multiple datasets for analysis

Data Cleaning

- **Data cleaning** is the preprocessing of **missing values** and removal data **noise** (outliers).
- **Pre-processing missing values**



- **Removing Noise**

Why the Need to Handle Missing Values?

- Missing values are generated by collection errors or missing observations.
- Missing values are the most common problem in data analytics.
- When missing values come out, certain model performance is decreased.
- Even for models that can handle missing values, they might be sensitive to it.
- Moreover, the model performance will be reduced with the increment of missing values such as data analytics on streaming data.

Missing Values

Column 0	age	years_seniority	income	parking_space	attending_party	entree	pets	emergency_contact
Tony	48	27		1	5	shrimp		Pepper
Donald	67	25	86	10	2	beef		Jane
Henry	69	21	95	6	1	chicken	62	Janet
Janet	62	21	110	3	1	beef		Henry
Nick		17		4				
Bruce	37	14	63		1	veggie		NA
Steve	83		77	7	1	chicken		n/a
Clint	27	9	118	9		shrimp	3	None
Wanda	19	7	52	2	2	shrimp		empty
Natasha	26	4	162	5	3			–
Carol		3	127	11	1	veggie	1	"""
Mandy	44	2	68	8	1	chicken		null

How do you deal with Missing Values?

- **Methods to process missing values**
  - Deletion
  - Dummy substitution
  - Mean substitution
  - Frequent substitution
  - Regression substitution

How do you deal with Missing Values?

- **Methods to process missing values**

Deletion: removing records with missing values

We can set a threshold for the ratio between number of missing elements and number of elements for a certain row to implement deletion.

Column 0	age	years_seniority	income	parking_space	attending_party	entree	pets	emergency_contact
Tony	48	27		1	5	shrimp		Pepper
Donald	67	25	86	10	2	beef		Jane
Henry	69	21	95	6	1	chicken	62	Janet
Janet	62	21	110	3	1	beef		Henry
Nick		17		4				
Bruce	37	14	63		1	veggie		NA
Steve	83		77	7	1	chicken		n/a
Clint	27	9	118	9		shrimp	3	None
Wanda	19	7	52	2	2	shrimp		empty
Natasha	26	4	162	5	3			-
Carol		3	127	11	1	veggie	1	****
Mandy	44	2	68	8	1	chicken		null

How do you deal with Missing Values?

- **Methods to process missing values**

Dummy substitution: Replace missing values with a dummy value e.g. **UNKNOWN** for category values or **0** for numerical values

We introduce additional attribute values, which will not significantly affect model performance if missing values don't dominate these attributes

Column 0	age	years_seniority	income	parking_space	attending_party	entree	pets	emergency_contact
Tony	48	27		1	5	shrimp		Pepper
Donald	67	25	86	10	2	beef	0	Jane
Henry	69	21	95	6	1	chicken	62	Janet
Janet	62	21	110	3	1	beef		Henry
Bruce	37	14	63		1	veggie		NA
Steve	83		77	7	1	chicken	0	n/a
Clint	27	9	118	9		shrimp	3	None
Wanda	19	7	52	2	2	shrimp		empty
Natasha	26	4	162	5	3		0	-
Carol		3	127	11	1	veggie	1	****
Mandy	44	2	68	8	1	chicken		null

How do you deal with Missing Values?

- **Methods to process missing values**

Frequent substitution: If the missing values are **categorical**, replace the missing values with the **most frequent item**.

For example, missing values of the attribute "**entree**" can be filled out with most frequent item "**chicken**".

Column 0	age	years_seniority	income	parking_space	attending_party	entree	pets	emergency_contact
Tony	48	27		1	5	shrimp	0	Pepper
Donald	67	25	86	10	2	beef	0	Jane
Henry	69	21	95	6	1	chicken	62	Janet
Janet	62	21	110	3	1	beef		Henry
Bruce	37	14	63		1	veggie	0	NA
Steve	83		77	7	1	chicken	0	n/a
Clint	27	9	118	9		shrimp	3	None
Wanda	19	7	52	2	2	shrimp	0	empty
Natasha	26	4	162	5	3	chicken	0	_
Carol	48	3	127	11	1	veggie	1	****
Mandy	44	2	68	8	1	chicken		null

How do you deal with Missing Values?

- **Methods to process missing values**

Mean substitution:

$$x_{mean} = \sum_{i=1}^{n} x_i / n$$

n: number of values

For **continuous** attributes, we could use **mean** of attribute values to replace the missing values.

Take the attribute "**Age**" as an example, we use the **mean** value **48** to replace the missing values for this attribute.

Column 0	age	years_seniority	income	parking_space	attending_party	entree	pets	emergency_contact
Tony	48	27		1	5	shrimp	0	Pepper
Donald	67	25	86	10	2	beef	0	Jane
Henry	69	21	95	6	1	chicken	62	Janet
Janet	62	21	110	3	1	beef		Henry
Nick		17		4				
Bruce	37	14	63		1	veggie	0	NA
Steve	83		77	7	1	chicken	0	n/a
Clint	27	9	118	9		shrimp	3	None
Wanda	19	7	52	2	2	shrimp	0	empty
Natasha	26	4	162	5	3	UNKNOW	0	-
Carol	48	3	127	11	1	veggie	1	****
Mandy	44	2	68	8	1	chicken		null

How do you deal with Missing Values?

- **Methods to process missing values**

Regression substitution: Use a regression method to replace the missing values with regressed values, if we have attributes with time-series features.

> Time-series data

> ARIMA [1] (autoregressive integrated moving average)

Column 0	age	years_seniority	income	parking_space	attending_party	entree	pets	emergency_contact
Tony	48	27		1	5	shrimp	0	Pepper
Donald	67	25	86	10	2	beef	0	Jane
Henry	69	21	95	6	1	chicken	62	Janet
Janet	62	21	110	3	1	beef		Henry
Nick		17		4				
Bruce	37	14	63		1	veggie	0	NA
Steve	83		77	7	1	chicken	0	n/a
Clint	27	9	118	9		shrimp	3	None
Wanda	19	7	52	2	2	shrimp	0	empty
Natasha	26	4	162	5	3	UNKNOW	0	-
Carol	48	3	127	11	1	veggie	1	****
Mandy	44	2	68	8	1	chicken		null

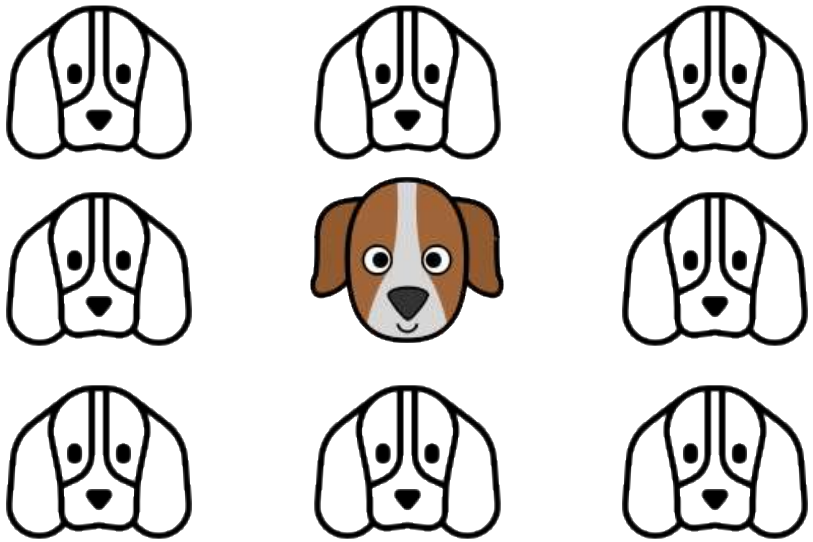
Comparison Table

Method	Data Loss	Accuracy	Best For
Deletion	High	Low–Medium	Small missing data
Dummy	No	Low	Special category meaning
Mean	No	Medium	Numeric data
Mode	No	Medium	Categorical data
Regression	No	High	Predictive modeling

Outliers

An **outlier** is a data point that differs significantly from other observations.

- Outliers can be very common in multidimensional data.
- Outliers can be results of bad data collection.
- Outliers would distort the models.
- Some models are sensitive to outliers.
- Sometimes outliers are the interesting data points.



How to deal with outliers?

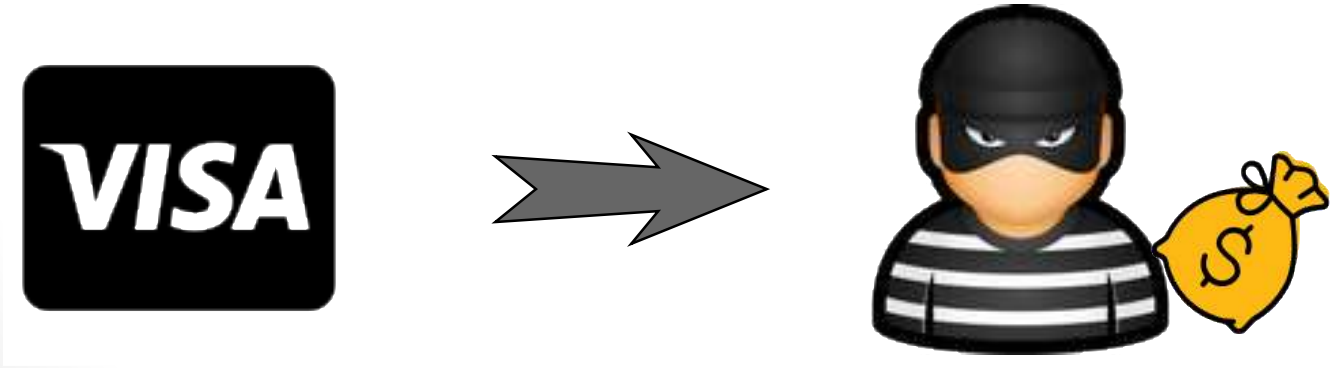
It depends on how to generate the outliers.

Keep outliers

We should pay more attention to the outliers since they may be genuine observations in the collected data.

In many applications, outliers provide **crucial information** for data Analytics

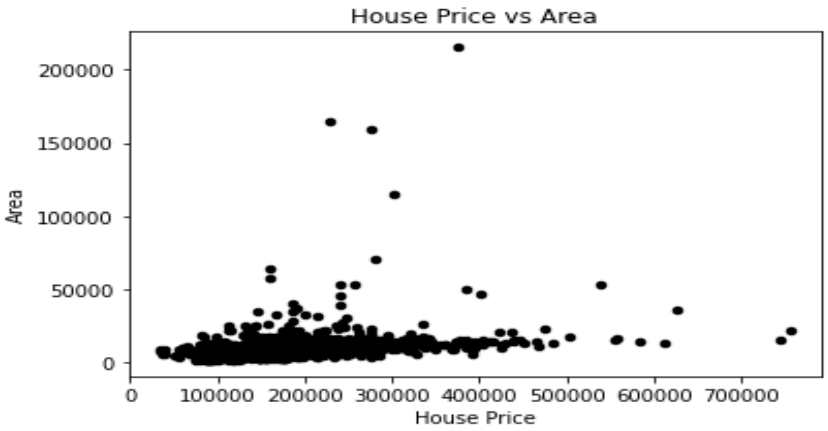
For example, credit card fraud detection alerts purchases that fall outside a customer's usual purchase patterns



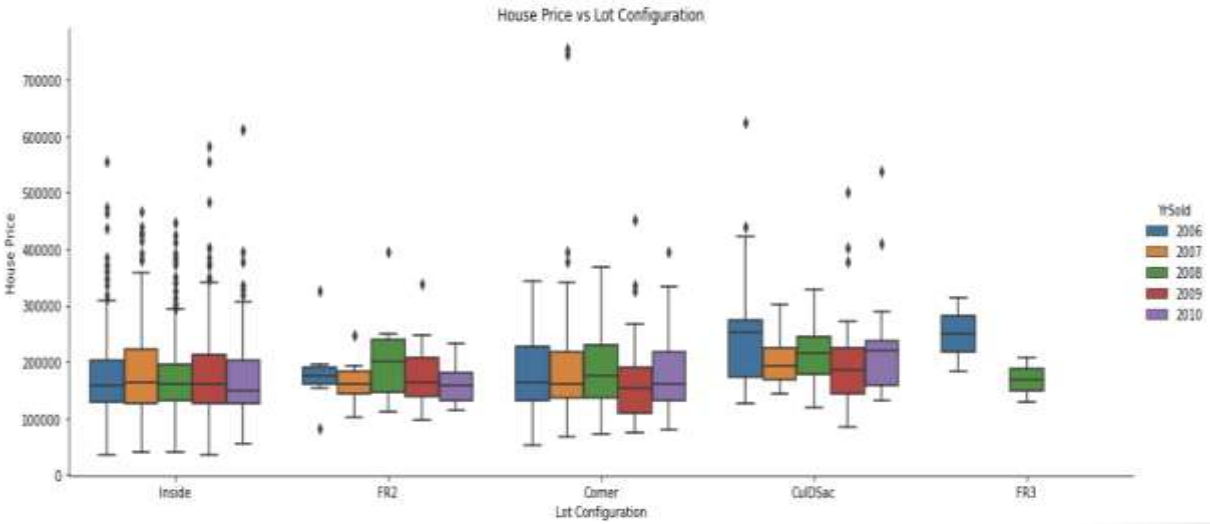
How to deal with outliers?

Two popular methods:

- **Scatter Plot**



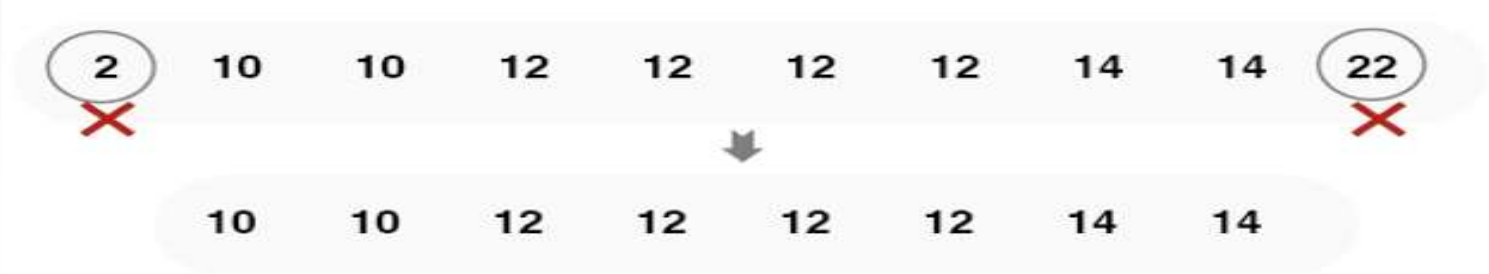
- **Box Plot**



How to deal with outliers?

Exclude outliers

Trimming: discarding the outliers.



Replacement: replacing the outliers with the nearest “normal” data point

Data Normalization

Data normalization is a process in which data attributes within a data model are organized to increase the cohesion of entity types.

In other words, the goal of data normalization is to **reduce** and even **eliminate data redundancy**.

There are two primary advantages of having a highly normalized data schema:

Increased consistency. Information is stored in one place and one place only, reducing the possibility of inconsistent data.

Easier object-to-data mapping. Highly-normalized data schemas in general are conceptually closer to object-oriented schemas because the object-oriented goals of promoting high cohesion and loose coupling between classes results in similar solutions (at least from a data point of view).

Data Normalization

Data normalization is the rescaling of numerical values to a specific range.

Two popular methods:

Min-Max Normalization:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Z-score Normalization (or Standardization)

$$\boldsymbol{X_{Z-score}} = \frac{\boldsymbol{X - \mu}}{\boldsymbol{\delta}}$$

$\boldsymbol{\mu}$: mean of X

δ: standard deviation of X

Data Down Sampling

Down sampling is the process of reducing the sampling rate of a signal, which is an effective way to handle imbalanced data.

Data down-sampling is reducing **large data to a smaller** and more manageable size.

- **Record down-sampling (clustering):** select the records and only choose the representative subset from the data.

- **Attribute down-sampling (Feature selection):** select only a subset of the most important attributes from the data.

Tasks

Go through OpenRefine “**Faceting**” at:
https://openrefine.org

Wrangler Video at:
https://vis.Stanford.edu/wrangler

(OR)

Go through “**Data Wrangling"** at:
https://www.alteryx.com/glossary/data-wrangling

Write down

Examples of “Data Dirtiness”
The above tools features

Data Wrangling

Imagine I am collecting the following data from you:

> Students' marks

> Attendance records

> Sports scores

But the data are:

> Messy

> In different forms

> Some values are missing

> Some names repeated, etc

Before you can calculate the required results, you must **Clean** it, **Transform** it, **Merge** it, **Reshape** it.This entire process is called "**Data Wrangling".**

Data Wrangling

Task: I want to prepare a final performance report of students using different datasets.

- **Create our data or load it from an external file**

```
import pandas as pd

import numpy as np

# Marks dataset

marks = pd.DataFrame({ "Student_ID": [101, 102, 103, 104],

"Name": ["Arun", "Bala", "Cathy", "Divya"], "Math": [90, 85, np.nan, 88], "Science": [92, 80, 75, np.nan] })

# Attendance dataset

attendance = pd.DataFrame({ "Student_ID": [101, 102, 103, 105], "Attendance%": [95, 88, 92, 85]

})

df = pd.DataFrame(marks)

df1 = pd.DataFrame(attendance)

print(df)

print(df1)
```

```
   Student_ID   Name  Math  Science
0         101   Arun  90.0     92.0
1         102   Bala  85.0     80.0
2         103  Cathy   NaN     75.0
3         104  Divya  88.0      NaN
   Student_ID  Attendance%
0         101           95
1         102           88
2         103           92
3         105           85
```

Data Cleaning

Data Cleaning – Handling Missing Data

> Check Missing Values marks.isnull()

> Fill Missing Values with Average or other values based on the requirements

```
marks["Math"].fillna(marks["Math"].mean(), inplace=True)
marks["Science"].fillna(marks["Science"].mean(), inplace=True)
```

inplace=True means "update the same DataFrame instead of creating a new one.

> Remove Rows with Missing Data (Alternative)

marks.dropna()

Data Transformation

Imagine Teacher wants grades instead of marks.

> Let's convert marks to Grade.

```
def grade(score):
    if score >= 90:
        return "A"
    elif score >= 75:
        return "B"
    else:
        return "C"
marks["Math_Grade"] = marks["Math"].apply(grade)
```

> Now we transformed numbers → categories.

Combining and Merging Datasets

> We have Marks dataset, Attendance dataset

> We want One final report card.

Basic Merge (Like Joining Two Notebooks)

```
final_report = pd.merge(marks, attendance,
on="Student_ID", how="inner")
```

> **What does how="inner" mean?**

Only students present in **both lists**.

> **Merge on Index**

If **Student_ID** is index:

```
marks.set_index("Student_ID", inplace=True)

attendance.set_index("Student_ID", inplace=True)

merged_index = marks.merge(attendance, left_index=True,
    right_index=True)
```

Inner Join

Student_ID	Name
101	Arun
102	Bala
103	Cathy
104	Divya

Student_ID	Attendance
101	95
102	88
103	92
105	85

```
final_report = pd.merge(marks, attendance, on="Student_ID", how="inner")
```

Student_ID	Name	Attendance
101	Arun	95
102	Bala	88
103	Cathy	92

Left and right

final_report = pd.merge(marks, attendance, on="Student_ID", how="left")

Student_ID	Name	Attendance
101	Arun	95
102	Bala	88
103	Cathy	92
104	Divya	NaN

final_report = pd.merge(marks, attendance, on="Student_ID", how="right")

Student_ID	Name	Attendance
101	Arun	95
102	Bala	88
103	Cathy	92
105	NaN	85

Outer Join

- **final_report = pd.merge(marks, attendance, on="Student_ID", how="outer")**

Student_ID	Name	Attendance
101	Arun	95
102	Bala	88
103	Cathy	92
104	Divya	NaN
105	NaN	85

Join Type	**Description**
Inner Join	Only matching rows
Left Join	All rows from left table
Right Join	All rows from right table
Outer Join	All rows from both tables

Concatenate (Stacking Data)

> Imagine we have

Class A marks

Class B marks, we stack them vertically.

```
import pandas as pd
classA = pd.DataFrame({
    "Name": ["Arun", "Bala"],
    "Math": [90, 85]
})
classB = pd.DataFrame({
    "Name": ["Cathy", "Divya"],
    "Math": [78, 88]
})
all_students = pd.concat([classA, classB])
print(all_students)
```

Index	Name	Math
0	Arun	90
1	Bala	85
0	Cathy	78
1	Divya	88

Combining with Overlap

> Sometimes two teachers entered marks and same student appears twice.

```
import pandas as pd

teacher1 = pd.DataFrame({
   "Student_ID": [101, 102],
   "Math": [90, 85]
})

teacher2 = pd.DataFrame({
   "Student_ID": [101, 102],
   "Science": [92, 80]
})
combined = pd.merge(teacher1, teacher2, on="Student_ID")
print(combined)
```

Student_ID	Math	Science
101	90	92
102	85	80

- **Overlap is handled by matching common columns.**

- **In pandas (Python library), the default join type used in pd.merge() is inner join.**

Reshaping Data

Name	Math	Science
Arun	85	88
Priya	90	92

> Imagine marks look like this:

| Name | Math | Science |

But principal wants:

| Subject | Marks |

We can reshape it.

```
reshaped = marks.reset_index().melt(
    id_vars=["Student_ID", "Name"],
    value_vars=["Math", "Science"],
    var_name="Subject",
    value_name="Marks")
```

Name	Subject	Marks
Arun	Math	85
Arun	Science	88
Priya	Math	90
Priya	Science	92

> Now each row represents one subject mark.

melt() is used to **reshape data from wide format to long format**.

It **converts multiple columns into rows** so the data becomes easier to analyze, visualize, or process.

Pivoting (Long → Wide)

> Pivot is like rearranging columns.

```
pivoted = reshaped.pivot(
    index="Name",
    columns="Subject",
    values="Marks“ )
```

> **Final School Report**

```
final_report["Total"] = final_report["Math"] + final_report["Science"]
# Sort students
final_report.sort_values(by="Total", ascending=False)
```

Data Wrangling – examples

```
import pandas as pd
data = {'Name': ['Alice', 'Bob',
None, 'David'],
        'Age': [25, None, 30, 40],
        'City': ['New York',
'Paris', 'London', None]}
df = pd.DataFrame(data)
df_cleaned = df.dropna()
# Fill missing values with default
values
df_filled = df.fillna({'Name':
'Unknown', 'Age': df['Age'].mean(),
'City': 'Unknown'})
print(df_filled)
```

Data Transformation

```
#Applying Functions to Columns (apply(), map())
df['Age_Category'] = df['Age'].apply(lambda x:
'Young' if x < 30 else 'Old’)

#Changing Data Types (astype())
df['Age'] = df['Age'].astype(int)
# Convert float to integer

#Renaming Columns (rename())
df = df.rename(columns={'Name': 'Full_Name'})

#Creating New Columns (assign())
df = df.assign(Age_Squared=df['Age'] ** 2)
```

Merging data

Using merge() for SQL-like Joins

```
df1 = pd.DataFrame({'ID': [1, 2, 3],
'Name': ['Alice', 'Bob',
'Charlie']})
df2 = pd.DataFrame({'ID': [1, 2, 4],
'Score': [85, 90, 75]})
```

```
merged_df = pd.merge(df1, df2,
on='ID', how='left')  # Left join
```

Clean, Transform and Merge

```
import pandas as pd

# Creating two datasets
df1 = pd.DataFrame({'ID': [1, 2, 3], 'Name': ['Alice',
'Bob', None], 'Age': [25, 30, None]})
df2 = pd.DataFrame({'ID': [1, 2, 4], 'Score': [85, 90,
75]})

# Cleaning: Fill missing values
df1['Name'].fillna('Unknown', inplace=True)
df1['Age'].fillna(df1['Age'].mean(), inplace=True)

# Transforming: Add Age Category
df1['Age_Category'] = df1['Age'].apply(lambda x: 'Young'
if x < 30 else 'Old')

# Merging datasets
merged_df = pd.merge(df1, df2, on='ID', how='left')

print(merged_df)
```

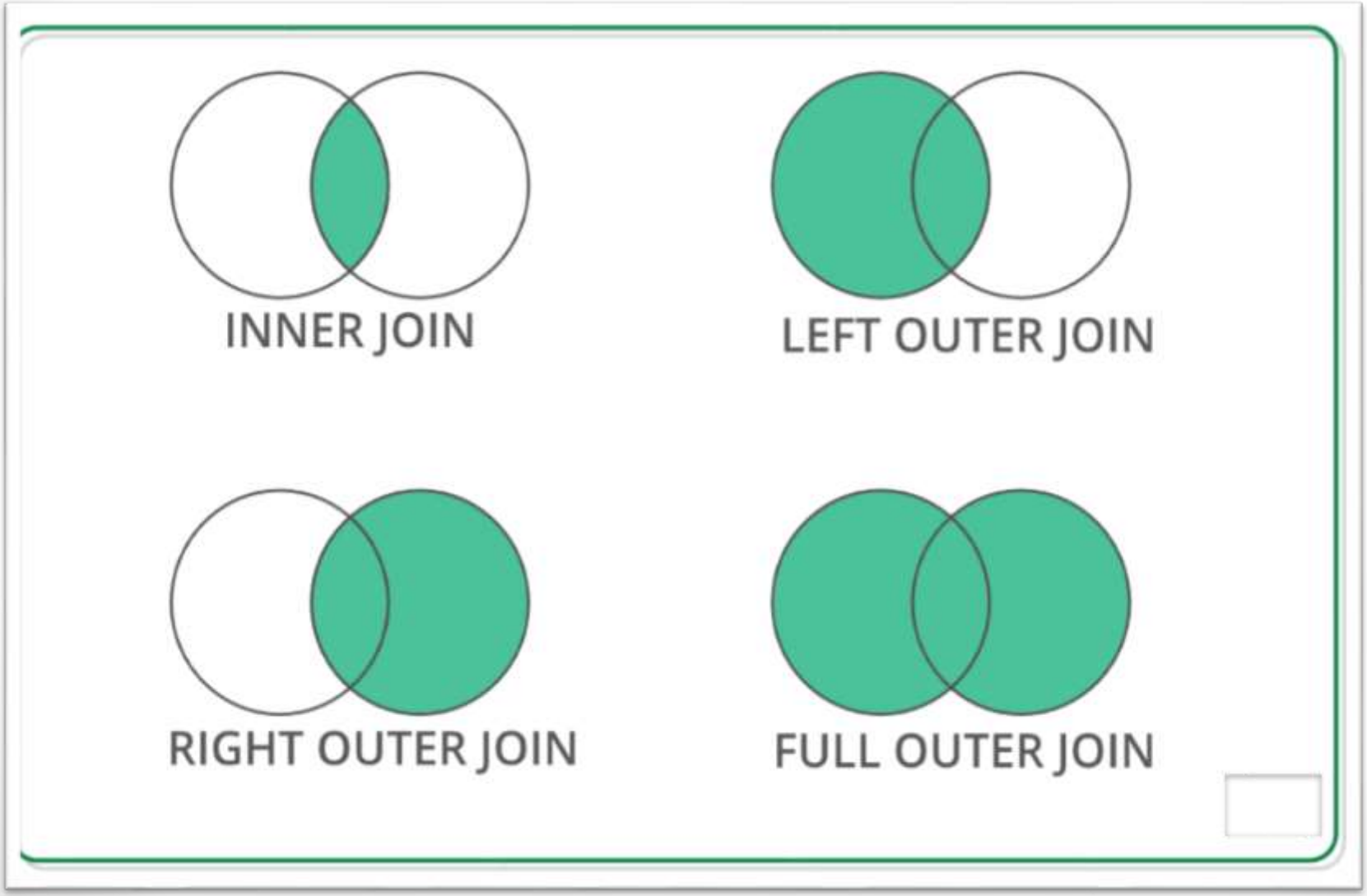
Combining and Merging datasets

- Data contained in Pandas objects can be combined together in a number of built-in ways:
- **pandas.merge** connects rows in DataFrames based on one or more keys. This will be familiar to users of SQL or other relational databases, as it implements database *join* operations.
- **pandas.concat** glues or stacks together objects along an axis.
- **combine_first** instance method enables splicing together overlapping data to fill in missing values in one object with values from another.

Combining and Merging datasets

```
import pandas as pd

df1 = pd.DataFrame({'ID': [1, 2, 3],
'Name': ['Alice', 'Bob', 'Charlie’]})

df2 = pd.DataFrame({'ID': [1, 2, 4],
'Score': [85, 90, 75]})

merged_df = pd.merge(df1, df2, on='ID',
how='inner’)
# Inner join
print(merged_df)
```

Combining and Merging datasets



Combining and Merging datasets

Concatenating Datasets (concat())

- The **concat()** function is used to stack datasets either vertically (row-wise) or horizontally (column-wise).

#Row-wise concatenation (Horizontally)

```
df1 = pd.DataFrame({'ID': [1, 2], 'Name':
['Alice', 'Bob']})
df2 = pd.DataFrame({'ID': [3, 4], 'Name':
['Charlie', 'David']})
```

 # Stacks them vertically

```
concat_df = pd.concat([df1, df2],
ignore_index=True)
print(concat_df)
```

#OUTPUT

	ID	Name
0	1	Alice
1	2	Bob
2	3	Charlie
3	4	David

Combining and Merging datasets

Column-wise concatenation

```
df1 = pd.DataFrame({'ID': [1, 2],
'Name': ['Alice', 'Bob’]})


df2 = pd.DataFrame({'Score': [85,
90]})


# Joins column-wise
concat_df = pd.concat([df1, df2],
axis=1)
print(concat_df)
```

	ID	Name	Score
0	1	Alice	85
1	2	Bob	90

Joining datasets

Joining Datasets (join())

- Similar to merge(), but by default, it joins on the index.

```
df1 = pd.DataFrame({'Name': ['Alice',
'Bob', 'Charlie']}, index=[1, 2, 3])
df2 = pd.DataFrame({'Score': [85, 90,
75]}, index=[1, 2, 4])
# Joins based on index
joined_df= df1.join(df2,how='outer')
print(joined_df)
```

	Name	Score
1	Alice	85.0
2	Bob	90.0
3	Charlie	NaN
4	NaN	75.0

Merging on Index

- When merging datasets based on the index, you can use either merge() with **left_index**=True, **right_index=True or join(),** which is specifically designed for index-based joins.

```
# Merging on index
merged_df = pd.merge(df1, df2,
left_index=True, right_index=True,
how='outer')
print(merged_df)
```

	Name	Score
1	Alice	85.0
2	Bob	90.0
3	Charlie	NaN
4	NaN	75.0

Merging on Index

```
# Merging on index (The other way)
merged_df = df1.merge(df2,
left_index=True, right_index=True,
how='outer')print(merged_df)
```

	ID_x	Name	ID_y	Score
0	101	Alice	101	85
1	102	Bob	102	90
2	103	Charlie	104	75

Merging on Index - Example

Resetting Index Explicitly Before Merging

- If the datasets have an ID column instead of an index, you may need to set the index first.

```
df1 = pd.DataFrame({'ID': [101, 102, 103],
'Name': ['Alice', 'Bob', 'Charlie']})
df2 = pd.DataFrame({'ID': [101, 102, 104],
'Score': [85, 90, 75]})
# Setting ID as the index
df1.set_index('ID', inplace=True)
df2.set_index('ID', inplace=True)
```

	Name	Score
ID		
101	Alice	85.0
102	Bob	90.0
103	Charlie	NaN
104	NaN	75.0

```
merged_df = df1.merge(df2, left_index=True,
right_index=True, how='outer')
print(merged_df)
```

Concatenation

Concatenating DataFrames (concat())

- Concatenation allows stacking datasets vertically (row-wise) or horizontally (column-wise).

	ID	Name
0	1	Alice
1	2	Bob
2	3	Charlie
3	4	David

Vertical Concatenation (Row-wise)

```
import pandas as pd

df1 = pd.DataFrame({'ID': [1, 2], 'Name': ['Alice',
'Bob']})
df2 = pd.DataFrame({'ID': [3, 4], 'Name': ['Charlie',
'David']})

concat_df = pd.concat([df1, df2], ignore_index=True)
# Stacks them
print(concat_df)
```

📌 **Key points:**

- **`ignore_index=True` resets the index after stacking.**
- **By default, it stacks row-wise (`axis=0`).**

Concatenation

Concatenating DataFrames (concat())

- Concatenation allows stacking datasets vertically (row-wise) or horizontally (column-wise).

Horizontal Concatenation (Column-wise)

```
df1 = pd.DataFrame({'ID': [1, 2], 'Name': ['Alice',
'Bob']})
df2 = pd.DataFrame({'Age': [25, 30], 'City': ['NY',
'LA']})

concat_df = pd.concat([df1, df2], axis=1)
# Joins column-wise
print(concat_df)
```

	ID	Name	Age	City
0	1	Alice	25	NY
1	2	Bob	30	LA

📌 Key points:

- *axis=1 joins column-wise.*
- *If indices don't match, missing values (NaN) appear.*

Combining with Overlap

Combining with Overlap (combine_first())

- This method is used to fill missing values in one DataFrame with values from another.

```
df1 = pd.DataFrame({'ID': [1, 2, 3], 'Name': ['Alice', 'Bob', None], 'Score': [85, None, 75]})
df2 = pd.DataFrame({'ID': [1, 2, 3], 'Name': [None, 'Bobby', 'Charlie'], 'Score': [None, 90, None]})
# Filling missing values in df1 with df2
combined_df = df1.combine_first(df2)
print(combined_df)
```

	ID	Name	Score
0	1	Alice	85.0
1	2	Bob	90.0
2	3	Charlie	75.0

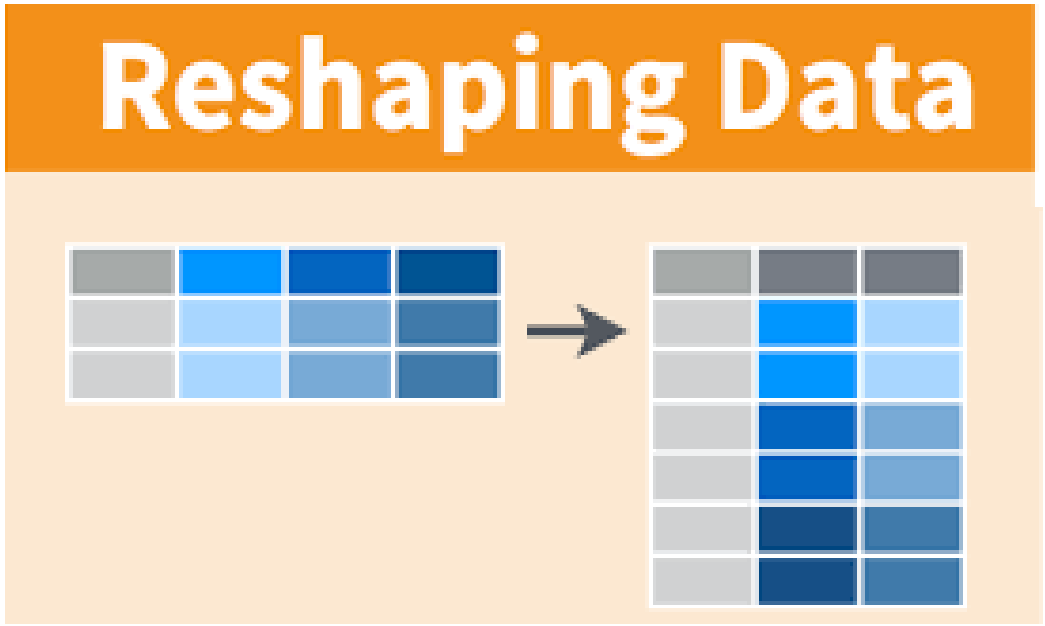
- 📌 **Key points:**
  - **It fills missing values (NaN) in df1 with values from df2 only if they exist.**
  - **Works element-wise, maintaining the structure.**

When to use which method?

Method	Use Case
concat()	Stacking datasets vertically (rows) or horizontally (columns)
merge()/join()	Joining datasets using a common column (like SQL joins)
combine_first()	Filling missing values in one dataset using another

Reshaping

- There are several fundamental operations for **rearranging** tabular data.
- These are alternatingly referred to as ***reshape*** or ***pivot*** operations.



Reshaping

Reshaping in pandas refers to changing the structure or layout of a DataFrame.

They are:

- **Pivoting**
- **Melting**
- **Stacking,**
- **Unstacking.**

Pivoting

Pivoting Data (pivot() and pivot_table())

- Pivoting rearranges data by turning unique values into columns.

Using pivot()

```
import pandas as pd
df = pd.DataFrame({
    'Date': ['2024-01-01', '2024-01-01', '2024-01-02', '2024-01-
02'],
    'City': ['New York', 'London', 'New York', 'London'],
    'Sales': [200, 150, 220, 180] })
```

```
pivot_df = df.pivot(index='Date',
columns='City', values='Sales’)
```

```
print(pivot_df)
```

City	London	New York
Date		
2024-01-01	150	200
2024-01-02	180	220

📌 **Key points:**

- **Converts City values into column names.**
- **Uses Date as the index.**
- **Shows Sales values as table entries.**

Melting

Melting Data (melt())

- The opposite of pivoting – it converts wide data into long format.

```
melted_df =
pivot_df.reset_index().melt(id_vars='Date',
var_name='City', value_name='Sales')
print(melted_df)
```

	Date	City	Sales
0	2024-01-01	London	150
1	2024-01-02	London	180
2	2024-01-01	New York	200
3	2024-01-02	New York	220

- 📌 **Key points:**
  - **Converts pivot_df back into a long format.**
  - **id_vars keeps specified columns (Date).**
  - **var_name and value_name rename the columns.**

Stacking and Unstacking

Stacking (stack())

- Converts **columns into a hierarchical index** (multi-index rows).

```
stacked_df = pivot_df.stack()
print(stacked_df)
```

City	London	New York
Date		
2024-01-01	150	200
2024-01-02	180	220

Unstacking (unstack())

- Converts multi-index rows back into columns.

```
unstacked_df=stacked_df.unstack()
print(unstacked_df)
```

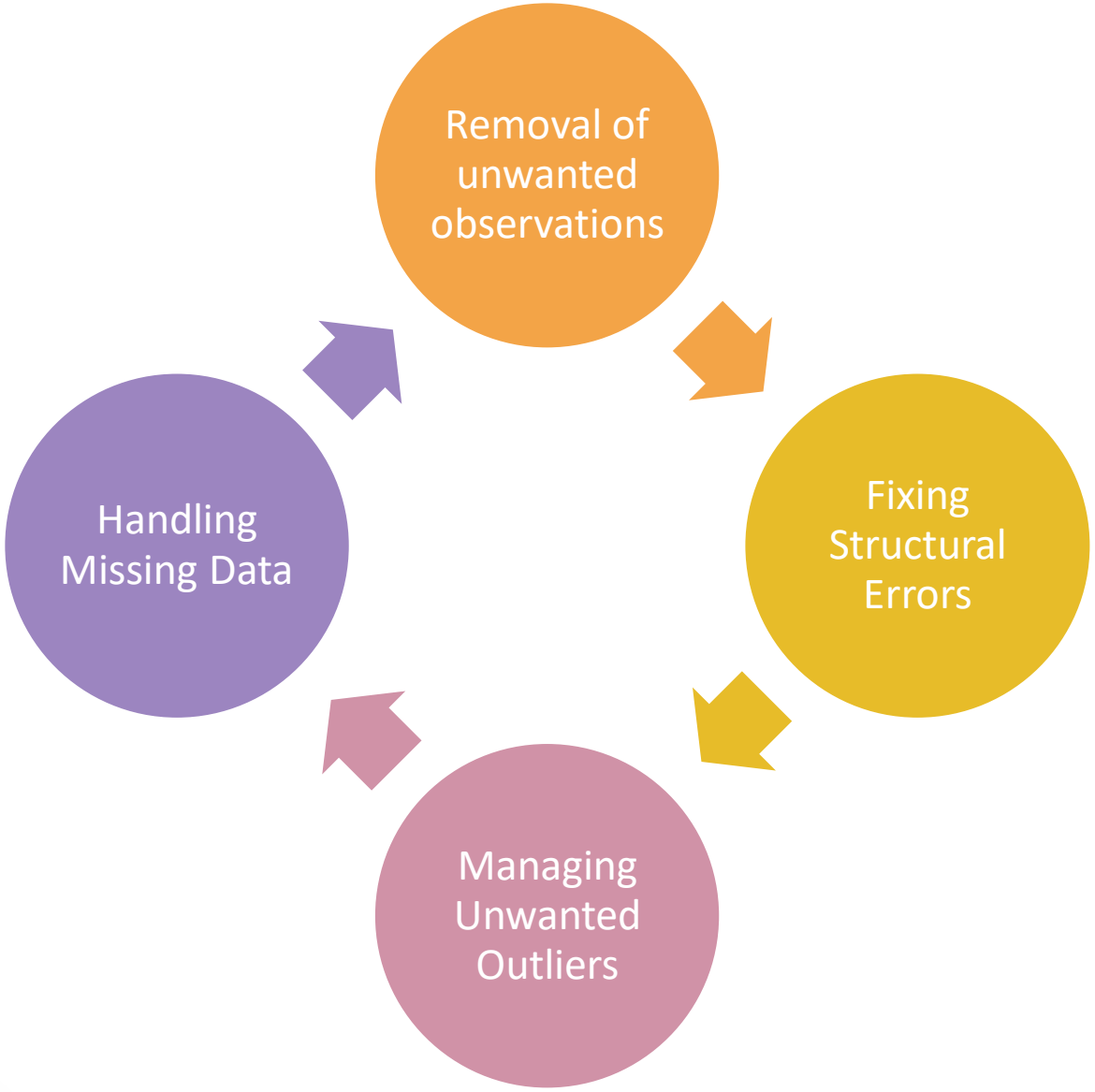
Data Cleaning

- Data cleaning, also known as data cleansing or data preprocessing, is a crucial step in the data science pipeline that involves identifying and correcting or removing errors, inconsistencies, and inaccuracies in the data to improve its quality and usability.
- Data cleaning is essential because raw data is often noisy, incomplete, and inconsistent, which can negatively impact the accuracy and reliability of the insights derived from it.

Why data cleaning is important?

- **Data cleansing** is a crucial step in the data preparation process, playing an important role in ensuring the accuracy, reliability, and overall quality of a dataset.
- For decision-making, the integrity of the conclusions drawn heavily relies on the cleanliness of the underlying data. Without proper data cleaning, inccuracies, outliers, missing values, and inconsistencies can compromise the validity of analytical results. Moreover, clean data facilitates more effective modeling and pattern recognition, as algorithms perform optimally when fed high-quality, error-free input.
- Additionally, clean datasets enhance the interpretability of findings, aiding in the formulation of actionable insights.

Steps to perform data cleanliness



Data Preparation

- **Data preparation** is the process of making raw data ready for after processing and analysis.
- The key methods are to collect, clean, and label raw data in a format suitable for machine learning (ML) algorithms, followed by data exploration and visualization.
- The process of cleaning and combining raw data before using it for machine learning and business analysis is known as data preparation.

Why data preparation is important?

- **Data preparation** ensures data accuracy and consistency, leading to reliable insights and informed decision-making.
- It optimizes data for analysis, uncovering hidden patterns and trends. Additionally, it enhances model performance and accuracy, driving better decision outcomes.
- It saves time and resources by preventing errors and inefficiencies in the analysis phase.

Steps to perform data preparation

Handling Missing data

What are missing values?

- Missing values are data points that are absent for a specific variable in a dataset.
- They can be represented in various ways, such as blank cells, null values, or special symbols like “NA” or “unknown.” These missing data points pose a significant challenge in data analysis and can lead to inaccurate or biased results.

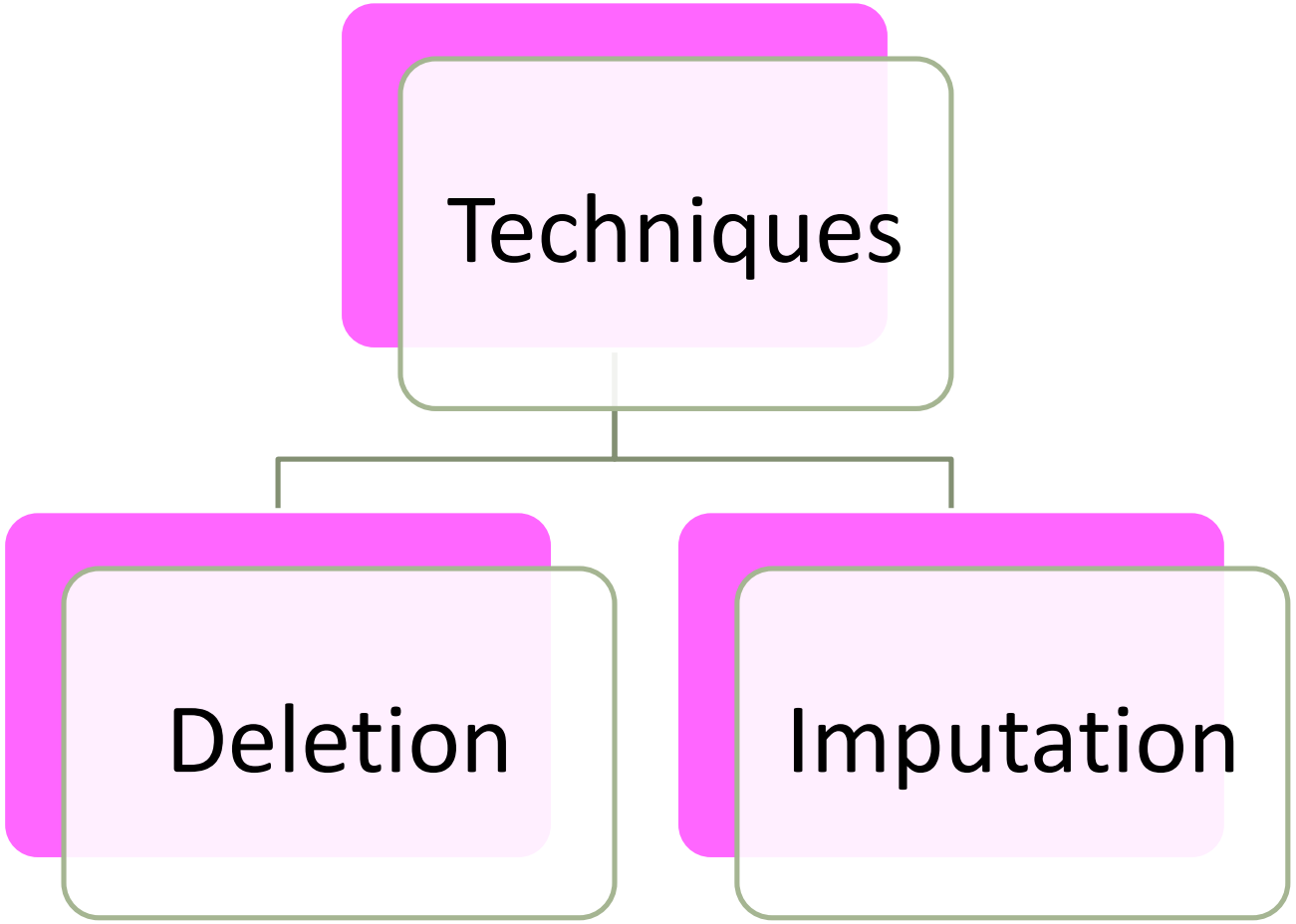
	School ID	Name	Address	City	Subject	Marks	Rank	Grade
0	101.0	Alice	123 Main St	Los Angeles	Math	85.0	2	B
1	102.0	Bob	456 Oak Ave	New York	English	92.0	1	A
2	103.0	Charlie	789 Pine Ln	Houston	Science	78.0	4	C
3	NaN	David	101 Elm St	Los Angeles	Math	89.0	3	B
4	105.0	Eva	NaN	Miami	History	NaN	8	D
5	106.0	Frank	222 Maple Rd	NaN	Math	95.0	1	A
6	107.0	Grace	444 Cedar Blvd	Houston	Science	80.0	5	C
7	108.0	Henry	555 Birch Dr	New York	English	88.0	3	B

Missing Values

Why is it important?

- Data Quality: Missing values can introduce errors and inconsistencies into your dataset, leading to inaccurate results and unreliable models.
- Model Performance: Many machine learning algorithms cannot handle missing values directly. If not addressed, they can either produce incorrect results or fail to converge.
- Bias: Improper handling of missing values can introduce bias into your analysis, leading to misleading conclusions. For example, if missing values are more likely to occur in certain groups, ignoring or deleting them can bias your results.
- Data Loss: Deleting rows or columns with missing values can result in significant data loss, especially if there are many missing values. This can reduce the statistical power of your analysis.
- Interpretation: Missing values can make it difficult to interpret the results of your analysis. For example, if a variable has many missing values, it may be difficult to draw meaningful conclusions about its relationship with other variables.

Techniques to handle missing data



Deletion

- Listwise Deletion: Remove entire rows or columns containing missing values. This method is simple but can result in a significant loss of data, especially if there are many missing values.
- Pairwise Deletion: Remove pairs of observations where at least one value is missing. This is less wasteful than listwise deletion but can introduce bias if missingness is not random.

Imputation

- Mean/Median/Mode Imputation: Replace missing values with the mean, median, or mode of the respective column. This is a simple approach but can introduce bias if the distribution is skewed.
- K-Nearest Neighbors (KNN) Imputation: Impute missing values using the average values of the k nearest neighbors. This method can be effective for numerical data.
- Regression Imputation: Use regression models to predict missing values based on other features. This is suitable for numerical data with strong relationships between features.
- Multiple Imputation: Create multiple imputed datasets by filling in missing values with different plausible values. This method can help to account for uncertainty in the imputation process.

Handling missing values

```
import pandas as pd
import numpy as np

# Creating a sample dataframe with missing values
data = {'Name': ['Alice', 'Bob', 'Charlie', 'David', np.nan],'Age': [25, np.nan, 30, 22, 35],'Salary': [50000, 54000, np.nan, 42000, 48000]}
df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
```

```
Original DataFrame:
      Name   Age   Salary
0    Alice  25.0  50000.0
1      Bob   NaN  54000.0
2  Charlie  30.0      NaN
3    David  22.0  42000.0
4      NaN  35.0  48000.0
```

Handling missing values

1)Check for missing values

```
print(df.isnull())
# Boolean DataFrame indicating
missing values
print(df.isnull().sum())
# Count of missing values in
each column
```

```
    Name    Age  Salary
0  False  False   False
1  False   True   False
2  False  False    True
3  False  False   False
4   True  False   False
Name      1
Age       1
Salary    1
dtype: int64
```

2)Removing missing values

```
#Drop rows with missing values
df_cleaned = df.dropna()
print(df_cleaned)
#Drop columns with
missing values
df_cleaned = df.dropna(axis=1)
print(df_cleaned)
```

```
    Name   Age   Salary
0  Alice  25.0  50000.0
3  David  22.0  42000.0
Empty DataFrame
Columns: []
Index: [0, 1, 2, 3, 4]
```

Handling missing values

3) Filling missing values

```
#Fill with a specific value
df_filled = df.fillna(0)
print(df_filled)
#Fill with column mean, median and mode
df['Age'].fillna(df['Age'].mean(),
inplace=True)  # Mean
df['Salary'].fillna(df['Salary'].median(),
inplace=True)  # Median
df['Name'].fillna(df['Name'].mode()[0],
inplace=True)  # Mode
print(df)
```

Handling missing values

3) Forward and backward fill

```
#Forward fill(propagates previous values
#forward)
  df_ffill = df.fillna(method='ffill')
  print(df_ffill)
  #Backward fill (propagates next value #backward)
  df_bfill = df.fillna(method='bfill')
  print(df_bfill)
```

Forward Fill

	Name	Age	Salary
0	Alice	25.0	50000.0
1	Bob	25.0	54000.0
2	Charlie	30.0	54000.0
3	David	22.0	42000.0
4	David	35.0	48000.0

Backward Fill

	Name	Age	Salary
0	Alice	25.0	50000.0
1	Bob	30.0	54000.0
2	Charlie	30.0	42000.0
3	David	22.0	42000.0
4	NaN	35.0	48000.0

Choosing the right approach

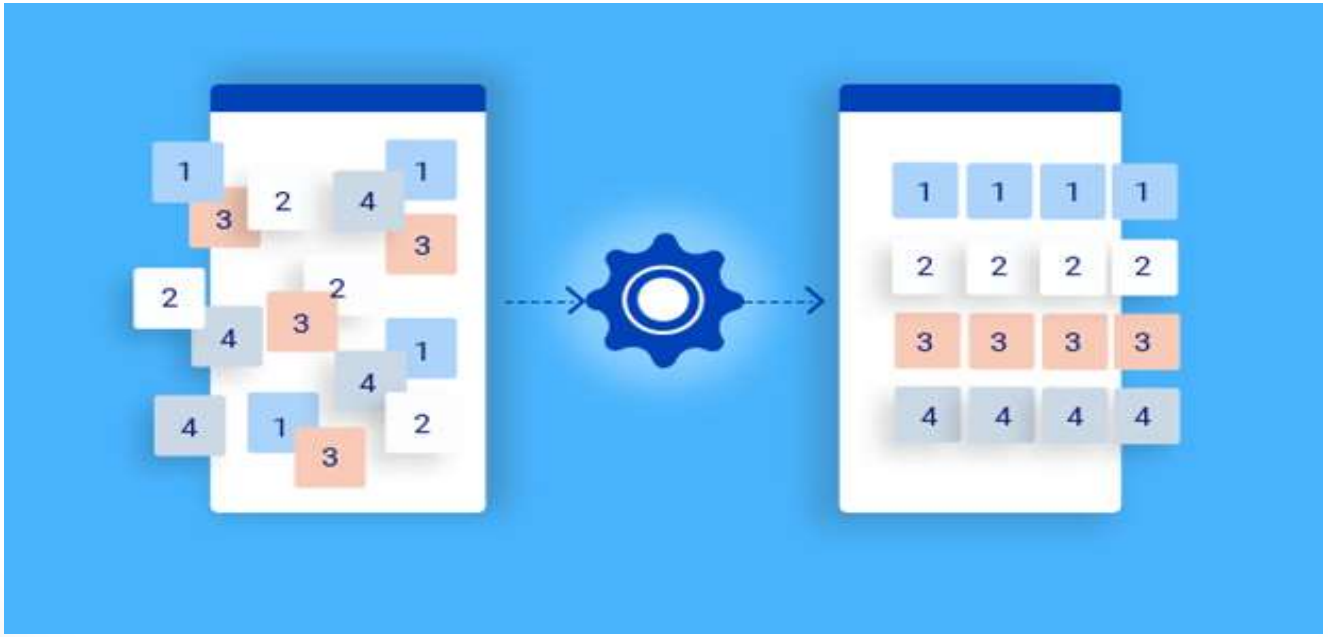
- The best approach for handling missing values depends on the nature of your data, the amount of missing data, and the specific requirements of your analysis. Consider the following factors:
  - Amount of missing data: If there are many missing values, imputation might be preferable to deletion.
  - Distribution of missing data: If missingness is random, imputation might be suitable. If missingness is related to other variables, more sophisticated techniques might be necessary.
  - Impact of missing data on the analysis: If missing values are likely to bias your results, it's important to address them.
- By carefully considering these factors and applying appropriate techniques, you can effectively handle missing values in your data science projects and improve the accuracy and reliability of your models.

Choosing the right approach

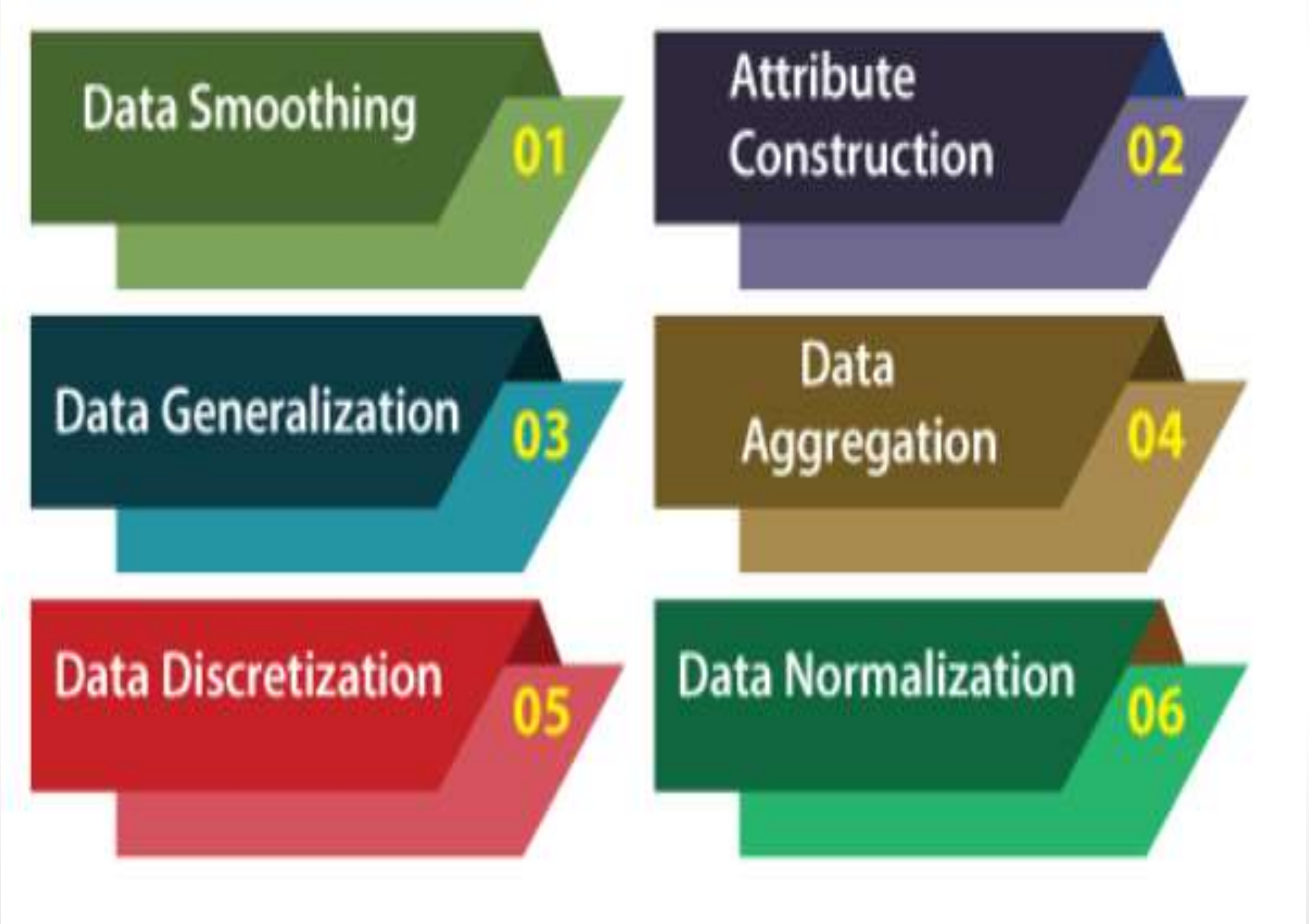
- **The best approach for handling missing values depends on the nature of your data, the amount of missing data, and the specific requirements of your analysis. Consider the following factors:**
  - **Amount of missing data: If there are many missing values, imputation might be preferable to deletion.**
  - **Distribution of missing data: If missingness is random, imputation might be suitable. If missingness is related to other variables, more sophisticated techniques might be necessary.**
  - **Impact of missing data on the analysis: If missing values are likely to bias your results, it's important to address them.**
- **By carefully considering these factors and applying appropriate techniques, you can effectively handle missing values in your data science projects and improve the accuracy and reliability of your models.**

Data Transformation

- **Data transformation** is the process of converting, cleansing, and structuring data into a usable format that can be analyzed to support decision making processes, and to propel the growth of an organization.
- Data transformation is used when data needs to be converted to match that of the destination system.



Data Transformation Techniques



Data Smoothing

- **Data smoothing** is a process that is used to **remove noise** from the dataset using some algorithms.
- It allows for highlighting important features present in the dataset.
- It helps in predicting the patterns.
- When collecting data, it can be manipulated to eliminate or reduce any variance or any other noise form.

Attribute Construction

- In the attribute construction method, the new attributes consult the existing attributes to construct a new data set that eases data mining.
- New attributes are created and applied to assist the mining process from the given attributes. This simplifies the original data and makes the mining more efficient.

Data Generalization

- Data generalization is the process of converting detailed data into a more abstract, higher-level representation while retaining essential information.
- It is commonly used in data mining, privacy preservation, and machine learning to reduce complexity and improve model generalization.
- Types
  - Attribute Generalization
  - Hierarchical Generalization
  - Numeric Generalization
  - Text Generalization

Data Generalization

1) Attribute Generalization

Replacing **specific values** with more **general categories**.

Example:

Raw Data: Age = {22, 25, 30, 35}

Generalized Data: Age Group = {20-30, 30-40}

2) Hierarchical Generalization

Using a **hierarchy** to replace specific values with broader categories.

Example:

Raw Data: "Toyota Corolla" → Generalized: "Toyota" →"Car"

Data Generalization

3) **Numeric Generalization**

Converting **continuous numerical** data into **categorical ranges.**

Example:

Raw Data: Salary = {45000, 52000, 61000}

Generalized: Salary Range = {40K-50K, 50K-60K, 60K-70K}

4) **Text Generalization**

Replacing **specific words** with **generalized versions.**

Example:

Raw Data: "John lives in New York"

Generalized: "Person lives in City"

Data Aggregation

- Data collection or aggregation is the method of storing and presenting data in a summary format.
- The data may be obtained from multiple data sources to integrate these data sources into a data analysis description.
- This is a crucial step since the accuracy of data analysis insights is highly dependent on the quantity and quality of the data used.

Data Discretization

- Data discretization is the process of **converting continuous numerical data into discrete categories (bins).**
- It is commonly used in machine learning, data mining, and feature engineering to simplify models and improve Interpretability.

```
# Sample dataset
data = {'Age': [22, 25, 30, 35, 40, 45, 50, 55, 60]}
df = pd.DataFrame(data)
 # Equal-width binning into 3 categories
df['Age_Binned'] = pd.cut(df['Age'], bins=3, labels=['Young', 'Middle-aged', 'Old’])
print(df)
```

Data Normalization

- **Data normalization** is a preprocessing technique used to **scale numerical data** into a specific range, usually [0,1] or [-1,1].
- It ensures that features contribute equally to a model, preventing bias due to different scales.

Why Normalize Data?

✅ Improves Machine Learning Performance – Many algorithms (e.g., KNN, SVM, Neural Networks) perform better with normalized data.
✅ Speeds Up Convergence – Gradient descent optimizes faster when features are scaled.
✅ Prevents Dominance of Large-Scale Features – Avoids a situation where one feature overpowers others.

String manipulation

- **Strings** are fundamental and essential data structures that every Python programmer works with.
- In Python, a string is a sequence of characters enclosed within either single quotes ('...') or doubles quotes ("...").
- It is an immutable built-in data structure, meaning once a string is created, **it cannot be modified**. However, we can create new strings by concatenating or slicing existing strings.

String manipulations

#Basic String Operations

```
# Defining a string
text = "Hello, Python!"
# String length
print(len(text))  # 14
# Accessing characters
print(text[0])  # 'H'
print(text[-1])  # '!'
# Slicing a string
print(text[0:5])  # 'Hello'
print(text[:5])   # 'Hello'
print(text[7:])   # 'Python!'
# Reversing a string
print(text[::-1])  # '!nohtyP ,olleH'
```

String manipulations

#String Case Manipulation

```
text = "hello python"

print(text.upper())     # 'HELLO PYTHON'
print(text.lower())     # 'hello python'
print(text.title())     # 'Hello Python'
print(text.capitalize())  # 'Hello python'
print(text.swapcase())    # 'HELLO PYTHON'
```

String manipulations

```
#String Concatenation and Repetition
str1 = "Hello"
str2 = "Python"

# Concatenation
print(str1 + " " + str2)  # 'Hello Python’

# Repetition
print(str1 * 3)  # 'HelloHelloHello'
```

String manipulations

 # String Searching and Replacing

```
text = "Python is fun"
# Check if substring exists
print("Python" in text)  # True
print("Java" not in text)  # True

# Find position
print(text.find("fun"))  # 10
print(text.index("is"))  # 7

# Replace substring
print(text.replace("fun", "awesome"))
# 'Python is awesome'
```

String manipulations

- **Splitting and Joining Strings**

```
text = "apple,banana,grape"
# Splitting a string into a list
fruits = text.split(",")
print(fruits)
# ['apple', 'banana', 'grape']

# Joining a list into a string
new_text = "-".join(fruits)
print(new_text)
# 'apple-banana-grape'
```

String manipulations

#Removing whitespaces

```
text = "  Python  "

print(text.strip())  # 'Python'
print(text.lstrip())  # 'Python  '
print(text.rstrip())  # '  Python'
```

String manipulations

#Formatting Strings

```
name = "Alice"
age = 25

# Using f-strings (Python 3.6+)
print(f"My name is {name} and I am {age} years old.")

# Using format()
print("My name is {} and I am {} years old.".format(name, age))

# Using % operator
print("My name is %s and I am %d years old." % (name, age))
```

String manipulations

#Checking String properties

```
text = "Python123"
print(text.isalpha())
#False(contains numbers)
print(text.isdigit())
#False(contains letters)
print(text.isalnum())
# True (letters and numbers only)
print(text.isspace())
# False (not just spaces)
print("hello".islower())  # True
print("HELLO".isupper())  # True
```

String manipulations

#Reversing words in a sentence

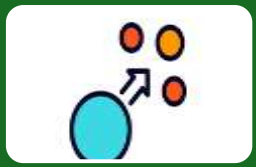
```
text = "Hello World"

reversed_words= " ".join(text.split([::-1])
print(reversed_words)  # 'World Hello'
```

Summarizing

- Data summarization is the process of reducing the complexity and size of data by extracting its main patterns, trends, and features.

Centrality

- Mean
- Median
- Mode



Dispersion

- Standard Deviation
- Variance
- Range

Sample Distribution

- Histogram
- Skewness
- Kurtosis

Summarizing

Centrality

- **Centrality of a data describes the center or middle value of the data set.**
- **The common measures of centrality are:**
  - **Mean: The average value of a dataset.**
  - **Mode: The most frequent value in a dataset.**
  - **Median: The middle value in a sorted dataset.**

Summarizing

Dispersion

- This category measures how spread out the data is.
- The common measures of dispersion are:
  - **Standard Deviation:** Measures the average distance of data points from the mean.
  - **Variance:** The square of the standard deviation.
  - **Range:** The difference between the maximum and minimum values.

Summarizing

Sample Distribution

- This category involves analyzing the shape and characteristics of the distribution of the data.
- The common methods under this category are:
  - **Histogram:** A graphical representation of the distribution of a numerical variable.
  - **Tally:** A simple counting method.
  - **Skewness:** Measures the asymmetry of the distribution.
  - **Kurtosis:** Measures the "tailedness" of the distribution.

Binning

- **Binning** is the process of grouping continuous values into discrete bins (intervals).
- It is commonly used in data preprocessing, feature engineering, and data visualization to reduce complexity and improve interpretability.

Binning

Types of Binning

1. **Equal-Width Binning**
   - Divides the data into equal-sized intervals.
   - Example: Ages (22, 25, 30, 35, 40) → Bins: [20-30], [30-40], etc.
2. **Equal-Frequency Binning (Quantile Binning)**
   - Each bin contains (almost) the same number of values.
   - Example: 5 salaries split into 3 bins, each with ~2 values.
3. **Custom Binning**
   - Manually defining bins based on domain knowledge.
   - Example: Categorizing income levels → Low, Medium, High.

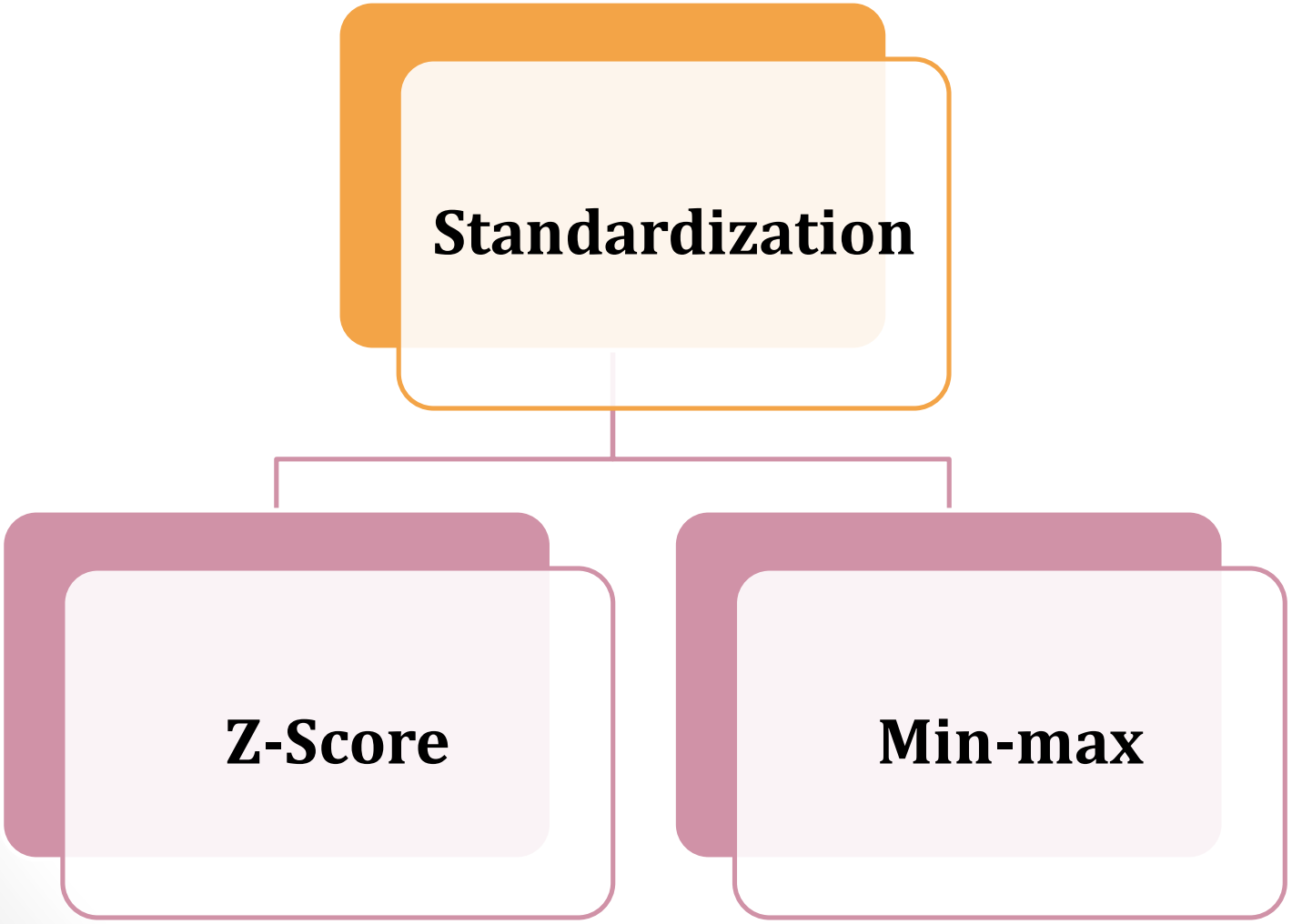
Standardization

- **Standardization** is a common preprocessing technique in data science that transforms numerical data to have a mean of 0 and a standard deviation of 1.
- This is particularly useful when dealing with features that have different scales or units, as it ensures that all features contribute equally to the model.

Why Standardization?

- Equalizes Feature Importance: Standardization prevents features with larger magnitudes from dominating the model, ensuring that all features are treated fairly.
- Improves Model Performance: Many machine learning algorithms, especially those based on distance or gradient calculations, benefit from standardized data.
- Compatibility with Certain Algorithms: Some algorithms, like K-Nearest Neighbors and Support Vector Machines, assume standardized data.

Standardization



Standardization

- **Z-score normalization** is a data preprocessing technique that transforms numerical data to have a mean of 0 and a standard deviation of 1. This is particularly useful when dealing with features that have different scales or units, as it ensures that all features contribute equally to the model.
- The formula used is:

  z = (x - mean) / standard_deviation

- **where:**
  - z is the normalized value.
  - x is the original value.
  - mean is the mean of the dataset.
  - standard_deviation is the standard deviation of the dataset.

Standardization

- **Min-max normalization** is a data preprocessing technique that scales numerical data to a specific range, typically between 0 and 1. It's useful when you want to preserve the relative distances between data points while ensuring that all features have a similar scale.
- The formula used is:

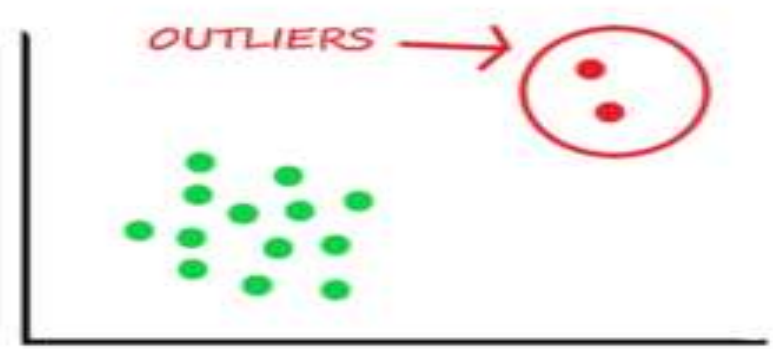
$$x_scaled = (x - min(x)) / (max(x) - min(x))$$

where:

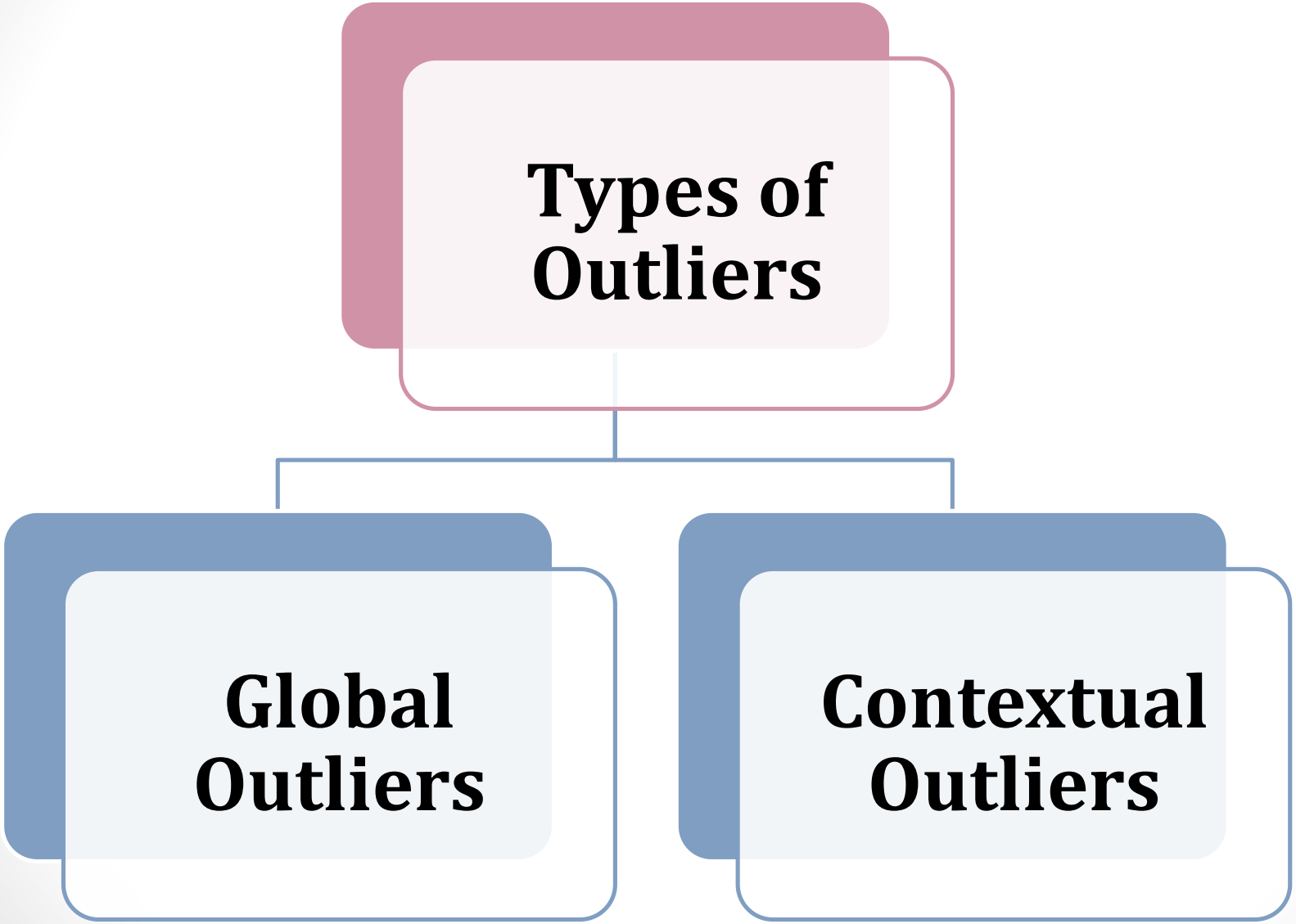
- x_scaled is the normalized value.
- x is the original value.
- min(x) is the minimum value in the dataset.
- max(x) is the maximum value in the dataset.

Outlier Noise

- Outlier Noise or Outliers are the data points which deviate significantly from the norm.
- Outliers can be single data points, or a subset of observations called a collective outlier.
- The outlier data points can greatly impact the accuracy and reliability of statistical analyses and machine learning models.
- Outliers can also be called abnormalities, discordant, deviants, or anomalies.



Outlier Noise



Types of Outliers

Global outliers

- Global outliers are isolated data points that are far away from the main body of the data.
- They are often easy to identify and remove.

Contextual outliers

- Contextual outliers are data points that are unusual in a specific context but may not be outliers in a different context.
- They are often more difficult to identify and may require additional information or domain knowledge to determine their significance.

Anomalies

- A data object that **deviates significantly** from normal objects, such as an unusual credit card purchase.
- Anomalies can also be instances or collections of data that are very rare in the data set and have features that differ significantly from most of the data.
- Anomalies can be **contextual outliers**, which are anomalies that have values that significantly deviate from other data points in the same context.
- For example, an anomaly in one dataset might not be an anomaly in another.